

Greedy Algorithms — Complete Guide

A comprehensive guide to greedy algorithms for placement interviews, competitive programming, and online assessments.

Sorting-Based Greedy

1. Overview

Sorting-based greedy is a class of greedy algorithms where the first step is to **sort the input** according to some key, and then make greedy choices in the sorted order. The sorting step imposes a structure that lets us make locally optimal decisions that lead to a globally optimal solution.

Many greedy problems are "sort and sweep" — sort by one criterion, then iterate making the best immediate choice.

2. Intuition

Simple explanation: When a problem involves items with start/end times, costs, deadlines, or weights, sorting organizes the data so that a single pass with a greedy rule suffices.

Analogy: Imagine scheduling classes in a classroom. If you sort classes by their ending time, you can always pick the class that finishes earliest, leaving maximum room for the rest.

Step-by-step reasoning:

1. What criterion should we sort by? (start time, end time, profit/weight ratio, deadline)
2. After sorting, can we make a decision at each step that never requires backtracking?
3. Prove that the sorting order preserves optimal substructure.

Why it works: When the optimal solution can be built by considering elements in a specific order (earliest finish time, highest ratio, smallest deadline), sorting lets us access elements in that order efficiently ($O(n \log n)$ instead of $O(n^2)$).

3. When to Use It

- Input has items with two attributes that need to be ordered
- Problems about scheduling, intervals, ratios, or pairings
- "Minimum number of X to cover Y" problems
- Problems where you intuitively want to "pick the best first"
- Trigger phrases: "*minimum number of platforms*", "*maximum profit*", "*schedule*", "*assign*", "*merge intervals*"

4. When Not to Use It

- When sorting does not reduce the problem to simple linear decisions

- When interactions between items are complex (decisions affect many later items)
- When the greedy choice after sorting fails (need DP instead)
- When the input is already in optimal order and you still can't solve greedily

5. Core Concepts

Concept	Explanation	Why It Matters
Comparator	A rule that defines ordering between two items	Determines which element the greedy algorithm considers "best" first
Canonical ordering	An ordering that respects optimal structure	If a problem has a canonical ordering, greedy works; otherwise it likely doesn't
Exchange argument	Prove that any optimal solution can be transformed to match the greedy order without worsening the result	The formal proof technique for sorting-based greedy
Stable vs unstable sort	Whether equal elements retain original order	Sometimes matters for tie-breaking

6. Step-by-Step Algorithm

1. **Identify** the sorting key (finish time, profit ratio, deadline, etc.)
2. **Sort** the input by this key using a custom comparator
3. **Initialize** result container (count, list, boolean array)
4. **Iterate** through sorted items:
 - Check if current item can be added (does it conflict? does it fit?)
 - If yes, add to result and update state
5. **Return** the result

7. Dry Run

Problem: Given intervals (start, end), select maximum non-overlapping intervals.

Input: [(1,3), (2,4), (3,6), (5,7), (6,8)]

Step 1: Sort by end time: [(1,3), (2,4), (3,6), (5,7), (6,8)]

Step	Current Interval	Last End	Pick?	Selected
1	(1,3)	$-\infty$	Yes (first)	[(1,3)]
2	(2,4)	3	No ($2 < 3$)	[(1,3)]
3	(3,6)	3	Yes ($3 \geq 3$)	[(1,3), (3,6)]
4	(5,7)	6	Yes ($5 \geq 6$)	[(1,3), (3,6), (5,7)]
5	(6,8)	7	Yes ($6 \geq 7$)	[(1,3), (3,6), (5,7), (6,8)]

Final answer: 4 intervals — but wait, (6,8) starts at 6, last ended at 7, $6 \geq 7$ is false! Correct result: 3 intervals [(1,3), (3,6), (6,8)].

Let me redo:

Step	Current Interval	Last End	Pick?	Selected
1	(1,3)	$-\infty$	Yes	[(1,3)]
2	(2,4)	3	No	[(1,3)]
3	(3,6)	3	Yes	[(1,3), (3,6)]
4	(5,7)	6	Yes	[(1,3), (3,6), (5,7)]
5	(6,8)	7 (=5's start < 7? No, 6 < 7)	No	[(1,3), (3,6), (5,7)]

Final: 3 intervals.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Interval {
    int start, end;
};

// Sort by end time (ascending), then by start time (ascending) for tie-breaking
bool cmp(const Interval &a, const Interval &b) {
    if (a.end != b.end) return a.end < b.end;
    return a.start < b.start;
}

vector<Interval> selectNonOverlapping(vector<Interval> &intervals) {
    if (intervals.empty()) return {};

    sort(intervals.begin(), intervals.end(), cmp);

    vector<Interval> result;
    result.push_back(intervals[0]);
    int lastEnd = intervals[0].end;

    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i].start >= lastEnd) {
            result.push_back(intervals[i]);
            lastEnd = intervals[i].end;
        }
    }
    return result;
}

int main() {
```

```

vector<Interval> intervals = {{1,3}, {2,4}, {3,6}, {5,7}, {6,8}};
vector<Interval> selected = selectNonOverlapping(intervals);
cout << "Selected " << selected.size() << " intervals:\n";
for (auto &iv : selected)
    cout << "(" << iv.start << "," << iv.end << ") ";
return 0;
}

```

9. Python Implementation

```

def select_non_overlapping(intervals):
    if not intervals:
        return []

    # Sort by end time
    intervals.sort(key=lambda x: (x[1], x[0]))

    result = [intervals[0]]
    last_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start >= last_end:
            result.append((start, end))
            last_end = end

    return result

# Example
intervals = [(1,3), (2,4), (3,6), (5,7), (6,8)]
selected = select_non_overlapping(intervals)
print(f"Selected {len(selected)} intervals: {selected}")

```

10. Code Explanation

- **Comparator `cmp`**: Defines ordering by end time first. This is crucial because the greedy choice is "pick the interval that finishes earliest".
- **`sort`**: $O(n \log n)$ preprocessing — the sorting step is what makes this a sorting-based greedy.
- **`lastEnd`**: Tracks the end time of the last selected interval. Each new interval is checked whether it starts after (or at) `lastEnd`.
- **Greedy choice**: Always pick the interval with the earliest finish time that doesn't conflict. This is locally optimal and leads to global optimum.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Single pass	$O(n)$

Operation	Complexity
Overall	$O(n \log n)$
Space (auxiliary)	$O(1)$ or $O(n)$ for output
Best case	$O(n \log n)$ (sorting dominates)
Worst case	$O(n \log n)$

12. Common Patterns

Pattern	How to Identify	Approach	Example
Earliest finish first	"Maximum non-overlapping intervals"	Sort by end time, pick if non-conflicting	Activity Selection
Highest ratio first	"Maximize value with weight constraint"	Sort by value/weight ratio	Fractional Knapsack
Smallest deadline first	"Schedule jobs before deadline"	Sort by deadline, assign slots	Job Sequencing
Earliest start first	"Cover all points"	Sort by start, merge/cover	Minimum Arrows to Burst Balloons

13. Common Mistakes

- Sorting by the wrong key (e.g., start time instead of end time for interval scheduling)
- Off-by-one in overlap check: using $>$ instead of $>=$ (or vice versa) for non-overlapping condition
- Forgetting to handle empty input
- Not defining a proper tie-breaking rule
- Using $O(n^2)$ approach after sorting instead of linear sweep

14. Edge Cases

- Empty input list
- Single interval
- All intervals overlap
- No intervals overlap
- All intervals have same start and end time
- Intervals with zero length (start == end)
- Back-to-back intervals (start == lastEnd)
- Large n (test $O(n \log n)$ performance)

15. Variations

Variation	What Changes	When Used
Sort by start then reverse	Sort descending by start	When we process from right to left

Variation	What Changes	When Used
Sort by length	Sort by interval length	When we want shortest intervals first
Multi-key sort	Sort by end, then start, then weight	When tie-breaking matters for optimality
Stable sort	Preserve original order for equal keys	When input order is a secondary criterion

16. Related Algorithms

Algorithm	Connection	How to Choose
Dynamic Programming	Also considers sequential decisions	Use greedy when local choice is always optimal; use DP when you need to explore choices
Two Pointers	Often used after sorting for $O(n)$ sweep	Two pointers for pair-based decisions; single pass for simple accumulation
Priority Queue	Alternative to sorting when dynamic ordering is needed	Sort when all items are known upfront; heap when new items arrive dynamically
Sweep Line Algorithm	Processes events at sorted coordinates	Use sweep line for more complex interval problems (counting overlaps, union)

17. Practice Problems

Easy

Problem	Platform	Idea	Difficulty
Assign Cookies	LeetCode 455	Sort greed, assign smallest cookie that satisfies each child	★
Minimum Difference Between Highest and Lowest of K Scores	LeetCode 1984	Sort, sliding window of size k	★

Medium

Problem	Platform	Idea	Difficulty
Non-overlapping Intervals	LeetCode 435	Sort by end, count removals	★★
Minimum Number of Arrows to Burst Balloons	LeetCode 452	Sort by end, shoot at end coordinate	★★
Car Pooling	LeetCode 1094	Sort by location, track capacity changes	★★

Hard

Problem	Platform	Idea	Difficulty
Maximum Profit in Job Scheduling	LeetCode 1235	Sort by end, DP + binary search	☆☆☆
Minimum Initial Energy to Finish Tasks	LeetCode 1665	Sort by (actual - minimum) difference	☆☆☆
Course Schedule III	LeetCode 630	Sort by deadline, max-heap for durations	☆☆☆

18. Interview Explanation

"Sorting-based greedy works when the optimal ordering of elements is known in advance. The key insight is that we sort the input by a specific criterion — like finish time, profit ratio, or deadline — and then make a single linear pass with a greedy rule. For example, in interval scheduling, sorting by finish time and always picking the earliest-finishing non-conflicting interval yields the maximum number of intervals. The proof typically uses an exchange argument: any optimal solution can be transformed to match the greedy ordering without losing optimality. The trade-off is that sorting costs $O(n \log n)$, which is usually acceptable."

19. Revision Notes

- **Key idea:** Sort input, then greedy pass
- **Sorting key:** Must be carefully chosen (end time, ratio, deadline)
- **Proof technique:** Exchange argument — swap non-greedy choices for greedy ones
- **Complexity:** $O(n \log n)$ dominated by sorting
- **Common trap:** Wrong sorting key leads to wrong answer
- **Remember:** Not all problems that look greedy are greedy — prove or test with counterexamples

20. Final Cheat Sheet

When to Use	Main Operations	Complexity	Key Code Idea
Interval scheduling	Sort, sweep	$O(n \log n)$	<code>sort(by end); if(start >= lastEnd) pick</code>
Ratio optimization	Sort by value/weight	$O(n \log n)$	<code>sort(by ratio); take whole or fraction</code>
Deadline-based	Sort by deadline	$O(n \log n)$	<code>sort(by deadline); assign nearest slot</code>
Merge intervals	Sort by start	$O(n \log n)$	<code>sort(by start); if(overlap) merge</code>

Interval Scheduling

1. Overview

Interval scheduling is the problem of **selecting a maximum-size subset of non-overlapping intervals** from a given set. Each interval has a start time and an end time. The goal is to maximize the count of intervals selected such that no two intervals overlap.

This is the classic textbook greedy algorithm and the foundation for many scheduling problems.

2. Intuition

Simple explanation: If you want to attend as many meetings as possible, always pick the meeting that ends earliest. Then skip all meetings that overlap with it and repeat.

Analogy: You have one classroom and many lecture requests. To maximize the number of lectures you can hold, always schedule the one that finishes earliest — it leaves the most room for others.

Step-by-step reasoning:

1. If a meeting ends early, it blocks less of the timeline for other meetings
2. Picking the earliest-finishing meeting is always safe because any optimal solution can be adjusted to include it
3. After picking it, recursively solve the same problem on the remaining non-conflicting intervals

Why it works: The exchange argument proves that there exists an optimal solution that includes the interval with the earliest finish time. This makes the greedy choice "safe" (locally optimal → globally optimal).

3. When to Use It

- "Maximum number of non-overlapping intervals"
- "Maximum number of tasks/meetings/events that can be scheduled"
- Resource allocation with exclusive usage
- Trigger phrases: "can attend", "no overlap", "maximum number", "schedule", "meeting room"

4. When Not to Use It

- When intervals have **weights/profits** (Weighted Interval Scheduling — needs DP)
- When the goal is to **cover all points** rather than select intervals
- When intervals can be **preempted** (paused and resumed)
- When there are **multiple resources/rooms** (that's a different problem)

5. Core Concepts

Concept	Explanation	Why It Matters
Non-overlapping	Two intervals don't share any time point	Core constraint; defines feasibility
Earliest finish time (EFT)	Sorting criterion: end time ascending	The key to the greedy choice
Exchange argument	Formal proof that greedy choice is optimal	Validates the algorithm mathematically

Concept	Explanation	Why It Matters
Feasibility check	Verifying <code>start >= last_end</code>	Ensures no overlap with selected set

6. Step-by-Step Algorithm

- Sort all intervals by **end time** (ascending)
- Initialize `last_end = -∞, count = 0`
- For each interval in sorted order:
 - If `interval.start >= last_end`:
 - Select it
 - `count++`
 - `last_end = interval.end`
- Return `count`

7. Dry Run

Input: [(2,5), (1,3), (4,6), (3,4), (5,7)]

Sorted by end: [(1,3), (3,4), (2,5), (4,6), (5,7)]

Step	Interval	last_end	start >= last_end?	Action	Selected
1	(1,3)	-∞	Yes	Select, last_end=3	1
2	(3,4)	3	Yes (3>=3)	Select, last_end=4	2
3	(2,5)	4	No (2<4)	Skip	2
4	(4,6)	4	Yes (4>=4)	Select, last_end=6	3
5	(5,7)	6	No (5<6)	Skip	3

Result: 3 intervals [(1,3), (3,4), (4,6)]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int maxNonOverlappingIntervals(vector<pair<int,int>> &intervals) {
    if (intervals.empty()) return 0;

    // Sort by end time
    sort(intervals.begin(), intervals.end(),
        [](auto &a, auto &b) { return a.second < b.second; });

    int count = 1;
    int lastEnd = intervals[0].second;

    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i].first >= lastEnd) {
```

```

        count++;
        lastEnd = intervals[i].second;
    }
}
return count;
}

int main() {
    vector<pair<int,int>> intervals = {{2,5}, {1,3}, {4,6}, {3,4}, {5,7}};
    cout << maxNonOverlappingIntervals(intervals); // 3
    return 0;
}

```

9. Python Implementation

```

def max_non_overlapping(intervals):
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1]) # Sort by end

    count = 1
    last_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start >= last_end:
            count += 1
            last_end = end

    return count

intervals = [(2,5), (1,3), (4,6), (3,4), (5,7)]
print(max_non_overlapping(intervals)) # 3

```

10. Code Explanation

- **Sorting by end:** The critical step. End time is the right criterion because it maximizes remaining availability.
- **last_end tracking:** Maintains the boundary of selected intervals. Any interval starting at or after this point is compatible.
- **Greedy pick:** The condition `start >= last_end` is the only check needed — no backtracking, no exploration.
- **Result:** The count equals the maximum cardinality subset of non-overlapping intervals.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Linear scan	$O(n)$
Overall	$O(n \log n)$
Space (auxiliary)	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Maximize count	"Maximum number of meetings/intervals"	Earliest finish time first
Minimum removals	"Minimum to remove to make non-overlapping"	Same as maximize selected (total - max selected)
One resource	Single room, single machine	Classic interval scheduling

13. Common Mistakes

- Sorting by start time instead of end time
- Using $>$ instead of $\geq$ for non-overlap (or vice versa depending on problem definition)
- Not handling $[(1,2), (2,3)]$ correctly (adjacent intervals may or may not count as overlapping)
- Forgetting to sort before the greedy pass

14. Edge Cases

- Empty list $\rightarrow 0$
- Single interval $\rightarrow 1$
- All overlapping $\rightarrow 1$
- All non-overlapping $\rightarrow n$
- Intervals with zero duration (start == end)
- Back-to-back intervals (end of one == start of next)

15. Variations

Variation	What Changes
Weighted Interval Scheduling	Each interval has a profit; maximize total profit. Needs DP with binary search
Interval Scheduling with Resources	Multiple machines/rooms available
Online Interval Scheduling	Intervals arrive in real-time, decisions made without future knowledge

16. Related Algorithms

Algorithm	Connection
Activity Selection	Same problem by a different name
Job Sequencing	Similar but with deadlines and profits
Minimum Platforms	Needs counting overlaps, not selecting non-overlapping
Meeting Rooms II	Minimum number of rooms needed = maximum depth of overlap

17. Practice Problems

Easy

Problem	Platform	Idea
Maximum Meetings in One Room	GFG	Classic interval scheduling
Non-overlapping Intervals	LeetCode 435	Total - max non-overlapping

Medium

Problem	Platform	Idea
Maximum Length of Pair Chain	LeetCode 646	Interval scheduling on pairs
Video Stitching	LeetCode 1024	Greedy on start-sorted intervals
Minimum Number of Arrows	LeetCode 452	Sort by end, shoot at end

Hard

Problem	Platform	Idea
Maximum Profit in Job Scheduling	LeetCode 1235	Weighted version, DP + binary search
Schedule with Minimum Cost	AtCoder	Complex constraints

18. Interview Explanation

"Interval scheduling asks: given intervals with start and end times, select the maximum number of non-overlapping intervals. The greedy approach sorts by end time ascending and always picks the first non-conflicting interval. The proof uses an exchange argument — any optimal solution can be transformed to include the interval with the earliest finish time, and the remaining problem is the same structure. This gives $O(n \log n)$ time and $O(1)$ extra space."

19. Revision Notes

- Sort by **end time** ascending
- Pick if `start >= last_end`
- Exchange argument for proof
- **$O(n \log n)$** time, **$O(1)$** space
- Total - max selected = minimum removals for non-overlap

20. Final Cheat Sheet

Use	Action	Time	Key Line
Max non-overlapping	Sort by end, greedily pick	$O(n \log n)$	<code>if(s >= lastEnd) { count++; lastEnd = e; }</code>

Activity Selection

1. Overview

Activity Selection is the same problem as Interval Scheduling under a different name. Given N activities with start and finish times, select the **maximum number of activities** that a single person can perform, assuming a person can work on only one activity at a time.

This is the classic greedy problem taught in virtually every algorithms course.

2. Intuition

Simple explanation: You have a list of activities, each with a start and end time. You can only do one at a time. Pick as many as possible by always choosing the one that ends earliest.

Analogy: Buffet dinner — you want to taste as many dishes as possible. Go to the dish that will be cleared away soonest (earliest end time), then move to the next.

Step-by-step reasoning:

1. Sort by finish time
2. Pick the first activity (smallest finish time)
3. Remove all activities that conflict with it
4. Repeat

Why it works: The problem has optimal substructure (the subproblem after picking an activity is also an activity selection problem) and the greedy choice property (picking the earliest-finishing activity is always optimal).

3. When to Use It

- "Maximum number of activities/tasks/jobs that can be done"
- Single resource scheduling
- Trigger phrases: "one person", "perform maximum", "non-overlapping"

4. When Not to Use It

- Activities have different **durations** and **profits** (weighted version $\rightarrow$ DP)
- Activities can be done **in parallel** with multiple resources
- Activities have **prerequisite relationships**

5. Core Concepts

Concept	Explanation	Why It Matters
Finish time	When the activity ends	Sorting key for greedy
Non-overlap constraint	<code>start_i >= finish_j</code> for consecutive activities	Defines feasibility
Greedy choice property	EFT choice leads to global optimum	What makes algorithm correct
Optimal substructure	Remaining problem after making a greedy choice is same type	Allows recursion/repetition

6. Step-by-Step Algorithm

1. Sort activities by finish time
2. Select the first activity
3. For each remaining activity:
 - If its start time $\geq$ finish time of last selected activity, select it

7. Dry Run

Input: (start, finish): (5,9), (1,2), (3,4), (0,6), (5,7), (8,9)

Sorted by finish: (1,2), (3,4), (0,6), (5,7), (5,9), (8,9)

i	Activity	Start	Finish	lastFinish	Start $\geq$ lastFinish?	Selected
1	A1	1	2	$-\infty$	Yes	✓
2	A2	3	4	2	Yes	✓
3	A3	0	6	4	No	✗
4	A4	5	7	4	Yes	✓
5	A5	5	9	7	No	✗
6	A6	8	9	7	Yes	✓

Result: 4 activities selected.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Activity {
    int start, finish;
};

int activitySelection(vector<int> start, vector<int> finish) {
    int n = start.size();
```

```

vector<Activity> acts(n);
for (int i = 0; i < n; i++) acts[i] = {start[i], finish[i]};

sort(acts.begin(), acts.end(),
     [](Activity &a, Activity &b) { return a.finish < b.finish; });

int count = 1, lastFinish = acts[0].finish;
for (int i = 1; i < n; i++) {
    if (acts[i].start >= lastFinish) {
        count++;
        lastFinish = acts[i].finish;
    }
}
return count;
}

int main() {
    vector<int> start = {5, 1, 3, 0, 5, 8};
    vector<int> finish = {9, 2, 4, 6, 7, 9};
    cout << activitySelection(start, finish); // 4
    return 0;
}

```

9. Python Implementation

```

def activity_selection(start, finish):
    n = len(start)
    activities = list(zip(start, finish))
    activities.sort(key=lambda x: x[1])

    count = 1
    last_finish = activities[0][1]

    for s, f in activities[1:]:
        if s >= last_finish:
            count += 1
            last_finish = f

    return count

start = [5, 1, 3, 0, 5, 8]
finish = [9, 2, 4, 6, 7, 9]
print(activity_selection(start, finish)) # 4

```

10–19. (Same structure as Interval Scheduling — Activity Selection is the same problem. Refer to Interval Scheduling sections for code explanation, complexity, patterns, mistakes, edge cases, variations, practice, interview explanation, revision notes, and cheat sheet.)

20. Final Cheat Sheet

Use	Action	Time	Key
Max activities	Sort by finish, greedy select	$O(n \log n)$	<code>if(s >= last_f) pick</code>

Minimum Platforms

1. Overview

Given arrival and departure times of trains at a station, find the **minimum number of platforms** required so that no train has to wait. This is equivalent to finding the **maximum number of trains present at the station at any time**.

2. Intuition

Simple explanation: Trains arrive and depart. When a train arrives, we need a free platform. When it departs, a platform becomes free. The minimum platforms needed equals the maximum number of trains simultaneously at the station.

Analogy: You're managing a parking lot. Cars arrive and leave. The number of parking spots you need is the maximum number of cars that are ever in the lot at the same time.

Step-by-step reasoning:

1. Separate arrivals and departures into two sorted lists
2. Merge them like a timeline: when an arrival happens, platform count goes up; when a departure happens, it goes down
3. Track the maximum of this count

Why it works: By sorting both events and processing them in time order, we can simulate exactly how many trains occupy the station at any moment. The peak of this occupancy curve is the minimum platforms needed.

3. When to Use It

- "Minimum number of platforms/rooms/resources required"
- "Maximum number of concurrent events/intervals"
- Overlap/congestion problems
- Trigger phrases: "*platform*", "*no train waits*", "*minimum rooms*", "*maximum overlap*"

4. When Not to Use It

- When you need to **schedule** rather than count resources (use interval scheduling)
- When resources are identical and assignment matters (different problem)
- When there are constraints on which resource can serve which interval

5. Core Concepts

Concept	Explanation	Why It Matters
Event-based processing	Treat arrivals (+1) and departures (-1) as events	Core technique for overlap counting
Peak concurrency	Maximum number of simultaneous intervals	The answer to minimum platforms
Sorted merge	Two-pointer technique on sorted arrays	$O(n \log n)$ instead of $O(n^2)$
Inclusive vs exclusive	When exactly does occupancy start/end	Handles trains arriving exactly as another departs

6. Step-by-Step Algorithm

1. Sort arrival times
2. Sort departure times
3. Initialize `platforms_needed = 1, max_platforms = 1`
4. Two pointers: `i = 1` (arrivals), `j = 0` (departures)
5. While `i < n`:
 - If `arr[i] <= dep[j]`: a train arrives before one departs → increment platforms, `i++`
 - Else: a train departs → decrement platforms, `j++`
 - Update `max_platforms = max(max_platforms, platforms_needed)`
6. Return `max_platforms`

7. Dry Run

Input:

Arrivals: [900, 940, 950, 1100, 1500, 1800]

Departures: [910, 1200, 1120, 1130, 1900, 2000]

Sorted arrays (already sorted):

Arr: [900, 940, 950, 1100, 1500, 1800]

Dep: [910, 1200, 1120, 1130, 1900, 2000]

i	j	arr[i]	dep[j]	arr[i] ≤ dep[j]?	Platforms	Max
1	0	940	910	No (940 > 910)	1-1=0, j=1	1
1	1	940	1200	Yes (940 ≤ 1200)	0+1=1, i=2	1
2	1	950	1200	Yes	1+1=2, i=3	2
3	1	1100	1200	Yes	2+1=3, i=4	3
4	1	1500	1200	No	3-1=2, j=2	3
4	2	1500	1120	No	2-1=1, j=3	3
4	3	1500	1130	No	1-1=0, j=4	3
4	4	1500	1900	Yes	0+1=1, i=5	3

i	j	arr[i]	dep[j]	arr[i] ≤ dep[j]?	Platforms	Max
5	4	1800	1900	Yes	1+1=2, i=6	3

Result: Minimum platforms needed = 3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int findPlatform(vector<int> &arr, vector<int> &dep) {
    int n = arr.size();
    sort(arr.begin(), arr.end());
    sort(dep.begin(), dep.end());

    int platforms = 1, maxPlatforms = 1;
    int i = 1, j = 0;

    while (i < n) {
        if (arr[i] <= dep[j]) {
            platforms++;
            i++;
        } else {
            platforms--;
            j++;
        }
        maxPlatforms = max(maxPlatforms, platforms);
    }
    return maxPlatforms;
}

int main() {
    vector<int> arr = {900, 940, 950, 1100, 1500, 1800};
    vector<int> dep = {910, 1200, 1120, 1130, 1900, 2000};
    cout << findPlatform(arr, dep); // 3
    return 0;
}
```

9. Python Implementation

```
def find_platform(arr, dep):
    arr.sort()
    dep.sort()

    n = len(arr)
    platforms = 1
    max_platforms = 1
    i, j = 1, 0
```

```

while i < n:
    if arr[i] <= dep[j]:
        platforms += 1
        i += 1
    else:
        platforms -= 1
        j += 1
    max_platforms = max(max_platforms, platforms)

return max_platforms

arr = [900, 940, 950, 1100, 1500, 1800]
dep = [910, 1200, 1120, 1130, 1900, 2000]
print(find_platform(arr, dep)) # 3

```

10. Code Explanation

- **Sort both arrays independently:** Unlike interval scheduling, we don't pair arrivals with departures. We sort them separately because each arrival just adds 1 and each departure subtracts 1.
- **Two pointers:** `i` traverses arrivals, `j` traverses departures. At any time, the difference between arrivals processed and departures processed is the current platform count.
- **Condition `arr[i] <= dep[j]`:** If a train arrives before or exactly when another departs, we need a new platform. The `<=` handles edge case where one arrives as another leaves.
- **maxPlatforms:** The running maximum of platforms needed.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Two-pointer scan	$O(n)$
Overall	$O(n \log n)$
Space (auxiliary)	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Minimum resources	"Minimum platforms/rooms/classrooms"	Sort arr/dep separately, two-pointer
Maximum concurrency	"Maximum customers/steps/overlap at any time"	Event +1 / -1 sweep
Meeting rooms II	Same as minimum platforms	Same algorithm

13. Common Mistakes

- Using `arr[i] < dep[j]` instead of `<=` — misses the case where arrival = departure
- Pairing arrival and departure (sorting by arrival then scanning) — wrong approach
- Not sorting both arrays (maintaining original pairs) — wrong
- Off-by-one in pointer logic

14. Edge Cases

- Single train $\rightarrow 1$
- All trains at same time $\rightarrow n$
- No trains $\rightarrow 0$
- Trains that arrive and depart at exactly the same time
- Decreasing order arrival times (need sorting)
- Very large time values

15. Variations

Variation	What Changes
Meeting Rooms II	Given meeting intervals, find minimum rooms. Same solution.
Car Pooling	Track capacity changes at each stop. Uses difference array.
My Calendar II	Track triple booking. More complex overlap counting.

16. Related Algorithms

Algorithm	Connection
Interval Scheduling	Different: selects intervals; Minimum Platforms counts overlap
Sweep Line	General technique; Minimum Platforms is a sweep line application
Difference Array	Alternative: mark +1 at arrival, -1 at departure, prefix sum

17. Practice Problems

Easy

Problem	Platform	Idea
Minimum Platforms	GFG	Classic train platform problem
Meeting Rooms (LeetCode 252)	LeetCode	Check if one room is enough

Medium

Problem	Platform	Idea
Meeting Rooms II (LeetCode 253)	LeetCode	Minimum rooms = minimum platforms
Car Pooling (LeetCode 1094)	LeetCode	Track passengers at each stop

Problem	Platform	Idea
My Calendar II (LeetCode 731)	LeetCode	Prevent triple booking

Hard

Problem	Platform	Idea
Minimum Number of Taps to Water Garden (LeetCode 1326)	LeetCode	Interval coverage, greedy
The Skyline Problem (LeetCode 218)	LeetCode	Complex sweep line

18. Interview Explanation

"Minimum Platforms uses a timeline sweep. We sort arrivals and departures separately, then traverse with two pointers. Each arrival increments the platform count, each departure decrements it. The maximum value of this counter is the minimum platforms needed. It's $O(n \log n)$ due to sorting, $O(1)$ space. The key insight is that arrivals and departures are independent events — we don't need to pair them."

19. Revision Notes

- Sort arrival and departure **independently**
- Two-pointer sweep, count = arrivals – departures
- Track **max count**
- Edge: $\leq$ for arrival $\leq$ departure
- **$O(n \log n)$ time, $O(1)$ space**

20. Final Cheat Sheet

When	Action	Time	Key
Min platforms / Max overlap	Sort arr & dep, two-pointer sweep	$O(n \log n)$	<code>if(arr[i] <= dep[j]) cnt++ else cnt--</code>

Meeting Rooms

1. Overview

Meeting Rooms problems ask: given a list of meeting intervals `[start, end]`, can a person attend **all** meetings (one room)? Or what is the **minimum number of conference rooms** needed (Meeting Rooms II)?

- **Meeting Rooms (LeetCode 252):** Check if one room is sufficient (no overlaps)
- **Meeting Rooms II (LeetCode 253):** Find minimum rooms needed

2. Intuition

Simple explanation for Meeting Rooms: Sort by start time. If any meeting starts before the previous one ends, you can't attend all.

Simple explanation for Meeting Rooms II: Same as Minimum Platforms — the maximum number of overlapping meetings at any time gives the minimum rooms needed.

Analogy: You have meetings scheduled. If two overlap, you need a second room. The maximum number that overlap at once is the rooms you need.

Step-by-step reasoning:

1. Sort meetings by start time
2. For Meeting Rooms: check if any `start < previous_end`
3. For Meeting Rooms II: sweep arrival/departure events, track max overlap

3. When to Use It

- "Can attend all meetings?"
- "Minimum meeting rooms / conference rooms"
- "Check for overlaps in intervals"
- Trigger phrases: "meeting", "conference room", "overlap", "attend all"

4. When Not to Use It

- When meetings have priorities/weights (different problem)
- When you need to schedule which meeting goes in which room (more complex assignment)
- When intervals have gaps that need filling

5. Core Concepts

Concept	Explanation	Why It Matters
Overlap detection	If <code>meeting[i].start < meeting[i-1].end</code> , they overlap	Core check for Meeting Rooms I
Chronological ordering	Sorting by start time	Makes overlap detection $O(n \log n)$
Concurrent count	Number of meetings active at a time	Answer for Meeting Rooms II
Event separation	Treat starts (+1) and ends (-1) as separate events	Foundation for sweep line

6. Step-by-Step Algorithm

Meeting Rooms I (Can attend all?)

1. Sort by start time
2. For $i = 1$ to $n-1$:
 - If `intervals[i].start < intervals[i-1].end`, return false
3. Return true

Meeting Rooms II (Minimum rooms)

1. Extract all start and end times

2. Sort both independently
3. Two-pointer sweep tracking concurrent count

7. Dry Run

Input: [(0,30), (5,10), (15,20)]

Meeting Rooms I

Sorted by start: [(0,30), (5,10), (15,20)]

i	Current	Previous	Check	Result
1	(5,10)	(0,30)	5 < 30 → overlap	false

Meeting Rooms II

Arrivals: [0, 5, 15]

Departures: [30, 10, 20]

Sorted arrivals: [0, 5, 15]

Sorted departures: [10, 20, 30]

i	j	arr[i]	dep[j]	arr[i] < dep[j]?	Count	Max
1	0	5	10	Yes	2	2
2	0	15	10	No	1	2
2	1	15	20	Yes	2	2
3	1	-	20	-	-	2

Result: 2 rooms

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Meeting Rooms I
bool canAttendAll(vector<vector<int>> &intervals) {
    sort(intervals.begin(), intervals.end());
    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] < intervals[i-1][1]) return false;
    }
    return true;
}

// Meeting Rooms II
int minMeetingRooms(vector<vector<int>> &intervals) {
    int n = intervals.size();
    vector<int> start(n), end(n);
```

```

for (int i = 0; i < n; i++) {
    start[i] = intervals[i][0];
    end[i] = intervals[i][1];
}
sort(start.begin(), start.end());
sort(end.begin(), end.end());

int rooms = 0, maxRooms = 0, i = 0, j = 0;
while (i < n) {
    if (start[i] < end[j]) {
        rooms++;
        i++;
    } else {
        rooms--;
        j++;
    }
    maxRooms = max(maxRooms, rooms);
}
return maxRooms;
}

int main() {
    vector<vector<int>> meetings = {{0,30}, {5,10}, {15,20}};
    cout << "Can attend all? " << canAttendAll(meetings) << "\n";
    cout << "Min rooms: " << minMeetingRooms(meetings); // 2
    return 0;
}

```

9. Python Implementation

```

def can_attend_all(intervals):
    intervals.sort(key=lambda x: x[0])
    for i in range(1, len(intervals)):
        if intervals[i][0] < intervals[i-1][1]:
            return False
    return True

def min_meeting_rooms(intervals):
    start = sorted(i[0] for i in intervals)
    end = sorted(i[1] for i in intervals)

    rooms = max_rooms = i = j = 0
    while i < len(intervals):
        if start[i] < end[j]:
            rooms += 1
            i += 1
        else:
            rooms -= 1
            j += 1
        max_rooms = max(max_rooms, rooms)
    return max_rooms

```

```
meetings = [[0,30], [5,10], [15,20]]
print(can_attend_all(meetings)) # False
print(min_meeting_rooms(meetings)) # 2
```

10. Code Explanation

- **Meeting Rooms I:** Sort by start, then a simple linear check. If any meeting starts before previous ends, overlap exists.
- **Meeting Rooms II:** Two-pointer sweep. Note the condition `start[i] < end[j]` (strict). When start equals end, it means one meeting ends as another begins — that doesn't require a new room.

11. Complexity Analysis

Problem	Time	Space
Meeting Rooms I	$O(n \log n)$	$O(1)$
Meeting Rooms II	$O(n \log n)$	$O(n)$

12–19. (See Minimum Platforms and Interval Scheduling sections for overlapping patterns.)

20. Final Cheat Sheet

Problem	Approach	Time	Key
Can attend all?	Sort by start, check overlap	$O(n \log n)$	<code>if(start[i] < end[i-1]) return false</code>
Min rooms	Sweep start/end events	$O(n \log n)$	<code>if(start[i] < end[j]) cnt++ else cnt--</code>

Fractional Knapsack

1. Overview

Given items with weights and values, and a knapsack of capacity W , find the **maximum value** you can carry. Unlike 0/1 Knapsack, you can take **fractions** of items. This makes the problem solvable greedily.

2. Intuition

Simple explanation: To maximize value in a limited-weight knapsack, always take the item with the **best value-to-weight ratio** first. Take as much as you can — if an item doesn't fit entirely, take a fraction of it.

Analogy: You're at a market with a bag that can hold 10 kg. You see gold (high value/kg), silver (medium), and sand (low). You'd fill your bag with as much gold as possible, then silver, then sand.

Step-by-step reasoning:

1. Calculate value/weight ratio for each item
2. Sort by ratio descending
3. Take whole items until one doesn't fit
4. Take a fraction of that item to fill the remaining capacity

Why it works: Since we can take fractions, sorting by ratio is optimal. Each unit of weight should be filled with the highest-value-per-unit-weight item available. This is a continuous optimization problem, not discrete.

3. When to Use It

- Items can be divided (liquids, powders, granular materials)
- "Maximum value with capacity constraint" + "fraction allowed"
- Trigger phrases: "*fractional*", "*weight to value ratio*", "*can be broken*"

4. When Not to Use It

- Items are **indivisible** (0/1 Knapsack) → use DP
- Items have **additional constraints** (minimum quantity, dependencies)
- Items have **non-linear value** (diminishing returns per unit)

5. Core Concepts

Concept	Explanation	Why It Matters
Value/weight ratio	Value per unit weight	Sorting key for greedy
Continuous selection	Can take any fraction of an item	What makes greedy work (unlike 0/1)
Remaining capacity	Weight still available	Decreases as we add items
Fractional filling	Taking a portion of the last item	Ensures full capacity usage

6. Step-by-Step Algorithm

1. Compute ratio = value/weight for each item
2. Sort items by ratio descending
3. Initialize `totalValue = 0`, `remainingW = W`
4. For each item:
 - If `item.weight ≤ remainingW`: take it whole, subtract weight, add value
 - Else: take fraction `remainingW / item.weight`, add proportionate value, break
5. Return `totalValue`

7. Dry Run

Input: $W = 50$

Items: (value, weight) = [(60, 10), (100, 20), (120, 30)]

Ratios: $60/10=6$, $100/20=5$, $120/30=4$

Sorted by ratio: [(60,10,r=6), (100,20,r=5), (120,30,r=4)]

Item	Value	Weight	Ratio	Remaining W	Take?	Value Added	Total Value
1	60	10	6	50	Whole	60	60
2	100	20	5	40	Whole	100	160
3	120	30	4	20	$20/30 = 2/3$	$120 \times 2/3 = 80$	240

Result: Maximum value = 240

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct Item {
    int value, weight;
};

double fractionalKnapsack(int W, vector<Item> &items) {
    sort(items.begin(), items.end(), [](Item &a, Item &b) {
        double r1 = (double)a.value / a.weight;
        double r2 = (double)b.value / b.weight;
        return r1 > r2;
    });

    double totalValue = 0.0;
    int remaining = W;

    for (auto &item : items) {
        if (item.weight <= remaining) {
            totalValue += item.value;
            remaining -= item.weight;
        } else {
            double fraction = (double)remaining / item.weight;
            totalValue += item.value * fraction;
            break;
        }
    }
    return totalValue;
}

int main() {
    int W = 50;
    vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};
    cout << fractionalKnapsack(W, items); // 240
    return 0;
}
```

9. Python Implementation

```
def fractional_knapsack(W, items):
    # items: list of (value, weight)
    items.sort(key=lambda x: x[0]/x[1], reverse=True)

    total_value = 0.0
    remaining = W

    for value, weight in items:
        if weight <= remaining:
            total_value += value
            remaining -= weight
        else:
            total_value += value * (remaining / weight)
            break

    return total_value

W = 50
items = [(60, 10), (100, 20), (120, 30)]
print(fractional_knapsack(W, items)) # 240.0
```

10. Code Explanation

- **Sort by ratio:** The greedy choice is the highest value per unit weight.
- **Whole vs fraction:** Take whole items if they fit; when one doesn't, take a fraction and stop.
- **Double for fraction:** Cast to double to avoid integer division.
- **Break after fraction:** After taking a fraction, knapsack is full.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Linear scan	$O(n)$
Overall	$O(n \log n)$
Space (auxiliary)	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Ratio greedy	Items with two attributes, one is limit	Sort by value/weight or similar ratio
Fractional selection	"Can take part of item"	Take whole items then fraction of last
Capacity filling	Maximize value under weight constraint	Fill with highest ratio first

13. Common Mistakes

- Using integer division for ratio comparison (use cross-multiplication or double)
- Not casting to double when computing fraction
- Sorting by value alone (ignores weight)
- Using this for 0/1 Knapsack (wrong algorithm)

14. Edge Cases

- $W = 0 \rightarrow 0$
- All items fit $\rightarrow$ sum of all values
- Single item, fraction needed
- Equal ratios (tie-breaking doesn't matter)
- Very large W (all items taken)
- Zero-weight items (avoid division by zero)

15. Variations

Variation	What Changes
0/1 Knapsack	Items cannot be divided. Use DP instead.
Bounded Knapsack	Limited copies of each item.
Unbounded Knapsack	Unlimited copies of each item.
Multiple Knapsacks	Multiple bags, divide items among them.

16. Related Algorithms

Algorithm	Connection
0/1 Knapsack (DP)	Same problem but items indivisible. Greedy fails.
Job Sequencing with Deadlines	Also uses ratio-based sorting (profit/duration).
Huffman Coding	Minimizes weighted path length using frequency-based greedy.

17. Practice Problems

Easy

Problem	Platform	Idea
Fractional Knapsack	GFG	Classic implementation
Maximum Units on a Truck (LeetCode 1710)	LeetCode	Fill truck with boxes of given units

Medium

Problem	Platform	Idea
Minimum Cost to Cut a Board into Squares	Codeforces	Greedy on costs, similar ratio concept

Problem	Platform	Idea
Maximum Value of K Coins from Piles (LeetCode 2218)	LeetCode	DP with greedy elements

Hard

Problem	Platform	Idea
Shopping Offers (LeetCode 638)	LeetCode	Combination of fraction-like offers
Minimum Cost to Connect Sticks (LeetCode 1167)	LeetCode	Greedy with min-heap

18. Interview Explanation

"Fractional Knapsack is the rare knapsack variant that's greedy. We compute value-to-weight ratio for each item, sort descending, and take as much as we can starting with the highest ratio item. Since items are divisible, we take whole items until the knapsack can't fit one fully, then take a fraction of the next item. This gives $O(n \log n)$ time. The proof is a simple exchange argument — if a higher-ratio item is left out, swapping it in improves the total."

19. Revision Notes

- Ratio = value / weight
- Sort by ratio descending
- Take whole → fraction of last
- **$O(n \log n)$** time
- Not for 0/1 Knapsack!
- Use double for fraction calculation

20. Final Cheat Sheet

Use	Action	Time	Key
Max value with divisible items	Sort by v/w ratio, greedy fill	$O(n \log n)$	<code>if(w <= W) take whole; else take fraction</code>

Jump Game

1. Overview

Jump Game (LeetCode 55): You are given an array where each element represents the **maximum jump length** from that position. Determine if you can reach the last index starting from index 0.

Jump Game II (LeetCode 45): Find the **minimum number of jumps** to reach the last index.

2. Intuition

Simple explanation: Think of "reachable range" — from your current position, you can reach any index up to $i + \text{nums}[i]$. The greedy approach is to keep track of the farthest reachable index.

Analogy: You're hopping across stepping stones in a river. Each stone tells you the maximum distance you can jump. You want to know if you can reach the far bank (last index). The greedy keeps extending the "farthest you can reach" as far as possible.

Step-by-step reasoning:

1. Track `maxReach` — the farthest index you can reach so far
2. Iterate through the array. At each index i :
 - If $i > \text{maxReach}$, you're stuck (unreachable)
 - Update $\text{maxReach} = \max(\text{maxReach}, i + \text{nums}[i])$
 - If $\text{maxReach} \geq n-1$, you can reach the end
3. For minimum jumps: jump only when reaching the end of the current jump's range

Why it works: The greedy maintains a frontier of reachable positions. For minimum jumps, you jump as late as possible within each jump's range (lazy jumping = greedy optimality).

3. When to Use It

- "Can you reach the end of array?"
- "Minimum jumps to reach end"
- Trigger phrases: "jump", "reach last index", "maximum steps", "minimum jumps"

4. When Not to Use It

- When jumps have **costs** that vary per jump (use DP)
- When you need the **path** not just count (needs tracking)
- When the array represents exact jumps (not max jumps) — simpler problem

5. Core Concepts

Concept	Explanation	Why It Matters
Max reachable index	Farthest position reachable from current path	Single value tracks all possibilities implicitly
Current jump range	$[\text{prevEnd}, \text{currEnd}]$ — indices reachable with current number of jumps	Used in Jump Game II to count jumps
Lazy jumping	Only increment jump count at the edge of current range	Minimizes jumps (greedy choice property)
Unreachable detection	When $i > \text{maxReach}$, position is unreachable	Early termination condition

6. Step-by-Step Algorithm

Jump Game I (Can reach?)

1. Initialize `maxReach = 0`
2. For `i = 0` to `n-1`:
 - If `i > maxReach`: return false
 - `maxReach = max(maxReach, i + nums[i])`
 - If `maxReach >= n-1`: return true
3. Return true

Jump Game II (Min jumps)

1. Initialize `jumps = 0`, `currEnd = 0`, `farthest = 0`
2. For `i = 0` to `n-2` (don't jump from last index):
 - `farthest = max(farthest, i + nums[i])`
 - If `i == currEnd`: must jump
 - `jumps++`
 - `currEnd = farthest`
3. Return `jumps`

7. Dry Run

Jump Game I

Input: `[2, 3, 1, 1, 4]`

i	nums[i]	i + nums[i]	maxReach	i > maxReach?	maxReach ≥ 4?
0	2	2	2	No	No
1	3	4	4	No	Yes → true

Result: true

Jump Game II

Input: `[2, 3, 1, 1, 4]`

i	nums[i]	farthest	i == currEnd?	jumps	currEnd
0	2	2	currEnd=0, Yes	1	2
1	3	4 (max=2,4)	1 ≠ 2, No	1	2
2	1	4 (max=4,3)	2 == 2, Yes	2	4
3	1	4 (max=4,4)	-	-	-

Result: 2 jumps

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

// Jump Game I
bool canJump(vector<int> &nums) {
    int maxReach = 0, n = nums.size();
    for (int i = 0; i < n; i++) {
        if (i > maxReach) return false;
        maxReach = max(maxReach, i + nums[i]);
        if (maxReach >= n - 1) return true;
    }
    return true;
}

// Jump Game II
int jump(vector<int> &nums) {
    int n = nums.size();
    if (n <= 1) return 0;

    int jumps = 0, currEnd = 0, farthest = 0;
    for (int i = 0; i < n - 1; i++) {
        farthest = max(farthest, i + nums[i]);
        if (i == currEnd) {
            jumps++;
            currEnd = farthest;
        }
    }
    return jumps;
}

int main() {
    vector<int> nums = {2, 3, 1, 1, 4};
    cout << "Can jump: " << canJump(nums) << "\n"; // 1
    cout << "Min jumps: " << jump(nums); // 2
    return 0;
}

```

9. Python Implementation

```

def can_jump(nums):
    max_reach = 0
    n = len(nums)
    for i in range(n):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + nums[i])
        if max_reach >= n - 1:
            return True
    return True

def jump(nums):
    n = len(nums)
    if n <= 1:

```

```
        return 0

    jumps = curr_end = farthest = 0
    for i in range(n - 1):
        farthest = max(farthest, i + nums[i])
        if i == curr_end:
            jumps += 1
            curr_end = farthest
    return jumps

nums = [2, 3, 1, 1, 4]
print(can_jump(nums)) # True
print(jump(nums))    # 2
```

10. Code Explanation

- **maxReach**: The greedy choice — from all positions reachable so far, what's the farthest index we can reach? This implicitly explores all paths.
- **i > maxReach**: If current index is beyond reachable range, we're stuck.
- **Jump Game II**: **farthest** tracks the best reach from current range. When we hit **currEnd**, we've exhausted the current jump's range and must jump.
- **n-1 loop limit in Jump Game II**: No need to jump from the last index.

11. Complexity Analysis

Problem	Time	Space
Jump Game I	O(n)	O(1)
Jump Game II	O(n)	O(1)

12. Common Patterns

Pattern	How to Identify	Approach
Reachability	"Can you reach the end?"	Track maxReach
Minimum jumps	"Minimum steps to reach end"	Lazy jumping (jump at range boundary)
Path with jumps	"Print the jump path"	Track nextIndex for each jump

13. Common Mistakes

- Forgetting that **nums[i]** is **maximum** jump, not exact
- Not handling single-element array (0 jumps for Jump Game II)
- Using BFS/DP unnecessarily (greedy works! O(n) vs O(n²))
- Off-by-one: looping **i < n** when **n-1** suffices

14. Edge Cases

- Single element → true (already at end), 0 jumps

- `[0]` → true, 0 jumps
- `[0, 1]` → false (stuck at index 0)
- `[1, 0, 0]` → false
- All zeros except first
- Large values (index overflow — use long long if needed)

15. Variations

Variation	What Changes
Jump Game III	Jump forward or backward by <code>nums[i]</code> . BFS needed.
Jump Game IV	Can teleport to same-valued indices. BFS.
Jump Game V	Can jump in either direction with height constraint. DP.
Minimum Jumps with Cost	Each jump has cost. DP needed.

16. Related Algorithms

Algorithm	Connection
BFS	Can solve minimum jumps ($O(n^2)$ worst case). Greedy is $O(n)$.
DP	Solves variants with costs; $O(n^2)$. Greedy is better when applicable.
Sliding Window	Jump Game's <code>[1, r]</code> range tracking is similar to sliding window.

17. Practice Problems

Easy

Problem	Platform	Idea
Jump Game (LeetCode 55)	LeetCode	Can reach last index
Minimum Number of Days to Make m Bouquets (LeetCode 1482)	LeetCode	Binary search, check feasibility similar to jump

Medium

Problem	Platform	Idea
Jump Game II (LeetCode 45)	LeetCode	Minimum jumps
Jump Game III (LeetCode 1306)	LeetCode	BFS with jump forward/back
Maximum Number of Coins You Can Get (LeetCode 1561)	LeetCode	Sorting + greedy picking

Hard

Problem	Platform	Idea
Jump Game IV (LeetCode 1345)	LeetCode	BFS with teleportation
Jump Game V (LeetCode 1340)	LeetCode	DP with monotonic stack

18. Interview Explanation

"Jump Game is a classic greedy problem. For the first variant, we maintain `maxReach` — the farthest index reachable from positions seen so far. If at any point the current index exceeds `maxReach`, return false. For the minimum jumps variant, we track the end of the current jump's range. When we reach it, we increment jump count and extend the range to the farthest reachable from within that range. Both are $O(n)$ time and $O(1)$ space. The key insight is that you never need to look back — the optimal jump at each step is to the position that gives the farthest reach."

19. Revision Notes

- **Jump Game I:** `maxReach = max(maxReach, i + nums[i])`. Return false if `i > maxReach`.
- **Jump Game II:** `farthest`, `currEnd`, `jumps`. Increment jumps at `currEnd`.
- Both $O(n)$ time, $O(1)$ space.
- Key: `nums[i]` is **maximum** jump, not exact.

20. Final Cheat Sheet

Problem	Greedy Choice	Time	Key Line
Can reach end?	Farthest reachable position	$O(n)$	<code>maxReach = max(maxReach, i + nums[i])</code>
Min jumps	Jump at end of current range	$O(n)$	<code>if(i == currEnd) { jumps++; currEnd = farthest; }</code>

Gas Station

1. Overview

Given two arrays `gas[i]` (fuel available at station i) and `cost[i]` (fuel needed to go from i to $i+1$), find the **starting station** such that you can complete a circular tour (visit all stations and return to start). Return -1 if impossible.

2. Intuition

Simple explanation: Start from station 0 with 0 fuel. Go station by station. At each station, `tank += gas[i] - cost[i]`. If tank ever goes negative, restart from the next station. If total gas $\geq$ total cost, exactly one solution exists.

Analogy: Driving a car around a circular track with gas stations. If you run out of gas between stations, that starting point doesn't work — try the next station. But also check if total gas is enough for the whole trip.

Step-by-step reasoning:

1. If $\text{sum}(\text{gas}) < \text{sum}(\text{cost})$, impossible $\rightarrow$ return -1
2. Otherwise, a solution exists. Start at station 0, track **tank**.
3. Whenever **tank** < 0 , reset **tank**=0 and try starting from **i+1**.
4. The last candidate start is the answer.

Why it works: If you fail at station j starting from i , then any start between i and j would also fail at j (because the tank would be even lower). So the next candidate is $j+1$.

3. When to Use It

- Circular tour / circuit problems
- "Starting point to complete journey"
- Trigger phrases: "gas station", "circuit", "circular tour", "starting point"

4. When Not to Use It

- When you can refuel only at some stations (different problem)
- When the graph is not a simple circle
- When there are multiple possible starting stations (modifications needed)

5. Core Concepts

Concept	Explanation	Why It Matters
Net gain	$\text{gas}[i] - \text{cost}[i]$ at each station	Core quantity; positive = gain, negative = loss
Total feasibility	$\text{sum}(\text{gas}) \geq \text{sum}(\text{cost})$	Necessary condition for any solution
Tank	Cumulative net gain from start	If negative, current start is invalid
Deficit skipping	If $\text{tank} < 0$ at station j , start over from $j+1$	Key greedy: any start between old_start and j also fails

6. Step-by-Step Algorithm

1. If $\text{sum}(\text{gas}) - \text{sum}(\text{cost}) < 0$, return -1
2. Initialize $\text{start} = 0, \text{tank} = 0$
3. For $i = 0$ to $n-1$:
 - $\text{tank} += \text{gas}[i] - \text{cost}[i]$
 - If $\text{tank} < 0$: $\text{start} = i + 1, \text{tank} = 0$
4. Return **start**

7. Dry Run

Input:

gas = [1, 2, 3, 4, 5]

cost = [3, 4, 5, 1, 2]

Net: [-2, -2, -2, 3, 3]

Total: $15 - 15 = 0 \geq 0 \rightarrow$ solution exists

i	gas[i]	cost[i]	gas-cost	tank before	tank after	tank < 0?	start
0	1	3	-2	0	-2	Yes	1
1	2	4	-2	0	-2	Yes	2
2	3	5	-2	0	-2	Yes	3
3	4	1	3	0	3	No	3
4	5	2	3	3	6	No	3

Result: Start at station 3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int canCompleteCircuit(vector<int> &gas, vector<int> &cost) {
    int n = gas.size();
    int totalGas = 0, totalCost = 0;
    for (int i = 0; i < n; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
    }
    if (totalGas < totalCost) return -1;

    int start = 0, tank = 0;
    for (int i = 0; i < n; i++) {
        tank += gas[i] - cost[i];
        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }
    return start;
}

int main() {
    vector<int> gas = {1, 2, 3, 4, 5};
    vector<int> cost = {3, 4, 5, 1, 2};
    cout << canCompleteCircuit(gas, cost); // 3
    return 0;
}
```

9. Python Implementation

```
def can_complete_circuit(gas, cost):
    if sum(gas) < sum(cost):
        return -1

    start = tank = 0
    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            start = i + 1
            tank = 0
    return start

gas = [1, 2, 3, 4, 5]
cost = [3, 4, 5, 1, 2]
print(can_complete_circuit(gas, cost)) # 3
```

10. Code Explanation

- **Total check first:** Quick rejection if total fuel isn't enough. Without this, we'd need more complex logic.
- **start = i + 1:** When tank goes negative, stations **start** through **i** are eliminated. The next candidate is **i+1**.
- **tank = 0:** Reset because we're starting fresh at the new candidate.
- **No second pass needed:** Since total gas $\geq$ total cost, the first surviving start is guaranteed to work.

11. Complexity Analysis

Operation	Complexity
Total check	$O(n)$
Single pass	$O(n)$
Overall	$O(n)$
Space	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Circular feasibility	"Complete circuit" "travel around"	Track net gain, reset on negative
Single pass elimination	"Starting point where cumulative never negative"	Greedy start candidate, reset on failure

13. Common Mistakes

- Not checking total gas vs cost first (leads to wrong answers in some edge cases)
- Resetting start to **i** instead of **i+1**
- Not resetting tank (using accumulated tank from previous failed start)

- Assuming need to simulate full circle after finding candidate (one pass suffices)

14. Edge Cases

- Single station: $\text{gas}[0] \geq \text{cost}[0] \rightarrow 0$, else -1
- All stations have zero net ($\text{gas} == \text{cost}$ exactly) $\rightarrow$ any station works, algorithm returns 0
- Total gas = total cost, but no station works? Not possible — at least one works.
- Very large gas/cost values (use long long for sum)

15. Variations

Variation	What Changes
Limited fuel tank capacity	Can't carry more than tank capacity. Need additional tracking.
Multiple laps	How many laps can you complete?
Weighted gas	Different fuel types, can only use matching type.

16. Related Algorithms

Algorithm	Connection
Maximum Subarray (Kadane)	Similar cumulative sum pattern, reset on negative
Jump Game	Also tracks range/reachability with greedy reset
Prefix Sum	Cumulative net gain analysis

17. Practice Problems

Easy

Problem	Platform	Idea
Gas Station (LeetCode 134)	LeetCode	Classic, single pass
Minimum Time to Complete Trips (LeetCode 2187)	LeetCode	Binary search on time

Medium

Problem	Platform	Idea
Circular Tour	GFG	Same as gas station
Minimum Fuel to Report to Capital	Codeforces	Tree-based fuel tracking

Hard

Problem	Platform	Idea
Minimum Refueling Stops (LeetCode 871)	LeetCode	Heap-based greedy for min refuel stops

Problem	Platform	Idea
Race Car (LeetCode 818)	LeetCode	BFS/DP, not directly related

18. Interview Explanation

"Gas Station is solved with a greedy single pass. First, check if total gas $\geq$ total cost — if not, impossible. Then maintain a tank variable and start candidate. As we traverse, if tank goes negative, we reset start to the next station. The key insight is that if you fail at station j starting from i , you'd also fail starting from any station between i and j , because your tank would be even more depleted. So we skip them and try $j+1$. $O(n)$ time, $O(1)$ space."

19. Revision Notes

- **Feasibility check:** `sum(gas)  $\geq$  sum(cost)` else -1
- **Single pass:** `tank += gas[i] - cost[i]`
- **Reset on negative:** `if(tank < 0) { start = i + 1; tank = 0; }`
- **$O(n)$ time, $O(1)$ space**
- **Key insight:** If start fails at j , all indices between start and j also fail

20. Final Cheat Sheet

Use	Action	Time	Key
Find starting gas station	Track tank, reset on negative	$O(n)$	<code>if(tank < 0) { start = i+1; tank = 0; }</code>

Assign Cookies

1. Overview

Given a list of children with greed factors (minimum cookie size they'll accept) and a list of cookies (each with a size), **maximize the number of content children**. Each child can get at most one cookie. A child is content if they receive a cookie of size $\geq$ their greed factor.

2. Intuition

Simple explanation: Give the smallest cookie that still satisfies each child. This way, larger cookies remain available for greedier children.

Analogy: You have different sizes of t-shirts and people of different sizes. To satisfy as many people as possible, give each person the smallest t-shirt that fits them. Don't give a large shirt to someone who only needs a small one.

Step-by-step reasoning:

1. Sort children by greed factor (ascending)
2. Sort cookies by size (ascending)

3. Try to satisfy the least greedy child with the smallest available cookie that works
4. If it works, move to next child and next cookie
5. If not, move to next cookie (too small for this child, certainly too small for greedier children)

Why it works: By matching smallest-sufficient cookie to the least greedy child, we conserve large cookies for greedier children. This is the classic "satisfy as many as possible" with minimal resource waste.

3. When to Use It

- "Maximum number of satisfied X with limited Y"
- Matching resources to demands with size constraints
- Trigger phrases: "*assign cookies*", "*content children*", "*satisfy*", "*greed factor*"

4. When Not to Use It

- When children can receive multiple cookies (different problem)
- When cookies have different values beyond size
- When there are other constraints (preferences, allergies)

5. Core Concepts

Concept	Explanation	Why It Matters
Greed factor	Minimum cookie size a child accepts	Sorting key for children
Sufficient cookie	$\text{cookie} \geq \text{child's greed factor}$	Condition for contentment
Smallest-satisfying	Give the smallest cookie that works	Conserves larger cookies
Two-pointer matching	Pointers on sorted children and cookies	Efficient $O(n + m)$ after sorting

6. Step-by-Step Algorithm

1. Sort greed factors ascending
2. Sort cookie sizes ascending
3. Initialize $i = 0$ (child index), $j = 0$ (cookie index)
4. While $i < n$ and $j < m$:
 - If $\text{cookie}[j] \geq \text{greed}[i]$: child i is content $\rightarrow i++, j++$
 - Else: cookie too small $\rightarrow j++$
5. Return i (number of content children)

7. Dry Run

Children (greed): [1, 2, 3]

Cookies (sizes): [1, 1]

Sorted children: [1, 2, 3]

Sorted cookies: [1, 1]

Step	Child	Cookie	Cookie $\geq$ Child?	Action	Content Count
1	1	1	Yes	Both advance	1
2	2	1	No ($1 < 2$)	Cookie advances	1
3	2	-	No more cookies	Stop	1

Result: 1 content child

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int findContentChildren(vector<int> &greed, vector<int> &cookies) {
    sort(greed.begin(), greed.end());
    sort(cookies.begin(), cookies.end());

    int i = 0, j = 0;
    while (i < greed.size() && j < cookies.size()) {
        if (cookies[j] >= greed[i]) {
            i++; // satisfied this child
        }
        j++; // try next cookie
    }
    return i;
}

int main() {
    vector<int> greed = {1, 2, 3};
    vector<int> cookies = {1, 1};
    cout << findContentChildren(greed, cookies); // 1
    return 0;
}
```

9. Python Implementation

```
def find_content_children(greed, cookies):
    greed.sort()
    cookies.sort()

    i = j = 0
    while i < len(greed) and j < len(cookies):
        if cookies[j] >= greed[i]:
            i += 1
        j += 1

    return i
```

```
greed = [1, 2, 3]
cookies = [1, 1]
print(find_content_children(greed, cookies)) # 1
```

10. Code Explanation

- **Sort both:** Sorting ensures we have access to smallest greed and smallest cookie in order.
- **while i < n && j < m:** Stop when either runs out.
- **cookies[j] >= greed[i]:** The smallest available cookie satisfies the least greedy remaining child. Both pointers advance.
- **Else → j++:** Cookie too small, skip it. Since children are sorted, a cookie too small for current child is too small for all remaining children.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n + m \log m)$
Two-pointer scan	$O(\min(n, m))$
Overall	$O(n \log n + m \log m)$
Space	$O(1)$

12. Common Patterns

Pattern	How to Identify	Approach
Matching smallest sufficient	"Assign resources to satisfy constraints"	Sort both, two-pointer match
Maximize count under threshold	"Max number of X that can be Y"	Greedy match smallest to smallest

13. Common Mistakes

- Matching largest cookie to greediest child (inefficient — wastes large cookies)
- Not sorting (need sorted order for two-pointer to work)
- Confusing which pointer to advance (both advance on match; only cookie advances on no-match)
- Returning j instead of i (i = content children count)

14. Edge Cases

- No children → 0
- No cookies → 0
- All children satisfied → i == n
- No cookies satisfy any child → 0
- All cookies identical
- Children with same greed factor

15. Variations

Variation	What Changes
Maximum happiness	Each cookie has a happiness value, maximize total happiness
Weighted assignment	Each child-cookie pair has a score, maximize sum
Multiple cookies per child	Child needs k cookies to be content

16. Related Algorithms

Algorithm	Connection
Sorting-based greedy	General pattern — sort and sweep
Two Pointers	Matching pattern
Hungarian Algorithm	Assignment with costs (much more complex)

17. Practice Problems

Easy

Problem	Platform	Idea
Assign Cookies (LeetCode 455)	LeetCode	Classic
Minimum Difference Between Highest and Lowest of K Scores (LeetCode 1984)	LeetCode	Sort + sliding window

Medium

Problem	Platform	Idea
Boats to Save People (LeetCode 881)	LeetCode	Two-pointer greedy with weight limit
Maximum Number of Coins You Can Get (LeetCode 1561)	LeetCode	Sort and take every second from end

Hard

Problem	Platform	Idea
Maximum Area of a Piece of Cake	LeetCode	After horizontal/vertical cuts

18. Interview Explanation

"Assign Cookies is a simple greedy matching problem. Sort children by greed factor and cookies by size, then use two pointers. For each cookie in ascending order, if it satisfies the current least-greedy

unsatisfied child, give it to them. This is optimal because giving a larger cookie than necessary would waste it. $O(n \log n + m \log m)$."

19. Revision Notes

- Sort both arrays ascending
- Match smallest cookie to smallest greed that it satisfies
- $O(n \log n + m \log m)$ time, $O(1)$ space
- Return i (children satisfied)

20. Final Cheat Sheet

Use	Action	Time	Key
Max content children	Sort both, two-pointer smallest-to-smallest	$O(n \log n)$	<code>if(cookie[j] >= greed[i]) i++; j++;</code>

Merge Intervals

1. Overview

Given an array of intervals `[start, end]`, **merge all overlapping intervals** and return an array of the non-overlapping intervals that cover all original intervals.

2. Intuition

Simple explanation: Sort intervals by start time. If the current interval overlaps with the last merged interval, merge them (extend the end). Otherwise, add it as a new interval.

Analogy: You have a set of time ranges when meetings are scheduled. If two meetings overlap in time, you can combine them into one block.

Step-by-step reasoning:

1. Sort by start time
2. Start with the first interval as the current merged interval
3. For each next interval:
 - If it starts before or at the current end → they overlap → merge by updating end to $\max(\text{end}, \text{current_end})$
 - Otherwise → no overlap → save the current merged interval and start a new one

Why it works: After sorting by start, overlapping intervals must appear consecutively. Any two overlapping intervals will be adjacent in the sorted order (or connected through a chain).

3. When to Use It

- "Merge overlapping intervals"
- "Combine intersecting ranges"
- Trigger phrases: "merge intervals", "overlap", "consolidate", "union of intervals"

4. When Not to Use It

- When you need to **count overlaps** (use sweep line)
- When intervals are **non-overlapping by definition** (merge is a no-op)
- When intervals have additional properties that affect merging (weights, categories)

5. Core Concepts

Concept	Explanation	Why It Matters
Overlap condition	<code>current.start ≤ last.end</code>	Determining if two intervals touch or overlap
Merging	<code>newInterval = [last.start, max(last.end, current.end)]</code>	Combining two overlapping intervals
Sorted adjacency	After sorting by start, overlapping intervals chain together	Key property that makes merge $O(n)$ after sort
Disjoint intervals	Non-overlapping intervals in the result	The output format

6. Step-by-Step Algorithm

1. Sort intervals by start time
2. Initialize `result` with the first interval
3. For $i = 1$ to $n-1$:
 - Let `last = result.back()`
 - If `intervals[i].start ≤ last.end`:
 - `last.end = max(last.end, intervals[i].end)`
 - Else:
 - Push `intervals[i]` to `result`
4. Return `result`

7. Dry Run

Input: `[[1,3], [2,6], [8,10], [15,18], [2,4]]`

Sorted by start: `[[1,3], [2,4], [2,6], [8,10], [15,18]]`

Step	Current	Last in Result	Overlap?	Action	Result
1	-	-	-	Add [1,3]	[[1,3]]
2	[2,4]	[1,3]	$2 \leq 3$: Yes	Merge $\rightarrow [1, \max(3,4)=4]$	[[1,4]]
3	[2,6]	[1,4]	$2 \leq 4$: Yes	Merge $\rightarrow [1, \max(4,6)=6]$	[[1,6]]
4	[8,10]	[1,6]	$8 \leq 6$: No	Add [8,10]	[[1,6], [8,10]]
5	[15,18]	[8,10]	$15 \leq 10$: No	Add [15,18]	[[1,6], [8,10], [15,18]]

Result: `[[1,6], [8,10], [15,18]]`

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> merge(vector<vector<int>> &intervals) {
    if (intervals.empty()) return {};

    sort(intervals.begin(), intervals.end());

    vector<vector<int>> result;
    result.push_back(intervals[0]);

    for (int i = 1; i < intervals.size(); i++) {
        auto &last = result.back();
        if (intervals[i][0] <= last[1]) {
            last[1] = max(last[1], intervals[i][1]);
        } else {
            result.push_back(intervals[i]);
        }
    }
    return result;
}

int main() {
    vector<vector<int>> intervals = {{1,3}, {2,6}, {8,10}, {15,18}, {2,4}};
    auto merged = merge(intervals);
    for (auto &iv : merged)
        cout << "[" << iv[0] << "," << iv[1] << " ] ";
    // [1,6] [8,10] [15,18]
    return 0;
}
```

9. Python Implementation

```
def merge(intervals):
    if not intervals:
        return []

    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]

    for start, end in intervals[1:]:
        last = merged[-1]
        if start <= last[1]:
            last[1] = max(last[1], end)
        else:
            merged.append([start, end])
```

```

    return merged

intervals = [[1,3], [2,6], [8,10], [15,18], [2,4]]
print(merge(intervals)) # [[1,6], [8,10], [15,18]]

```

10. Code Explanation

- **Sort by start:** The critical first step. C++ `sort` on vector of vectors sorts by first element by default.
- `last = result.back()`: Reference to the most recently added interval. We may modify it if merging.
- `intervals[i][0] <= last[1]`: Overlap check. The next interval overlaps if it starts before (or at) the last one ends.
- `last[1] = max(...)`: Extend the end time. We don't modify the start because start is always $\leq$ the current interval's start (due to sorting).
- **No overlap** $\rightarrow$ **push**: The interval is disjoint.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Linear scan	$O(n)$
Overall	$O(n \log n)$
Space	$O(n)$ for output

12. Common Patterns

Pattern	How to Identify	Approach
Interval merging	"Merge overlapping intervals"	Sort by start, merge if overlapping
Interval insertion	"Insert a new interval into sorted intervals"	Find position, merge neighbors
Overlap removal	"Remove minimum intervals to make non-overlapping"	Sort by end, count conflicts

13. Common Mistakes

- Not sorting first
- Only updating `last[1]` for `intervals[i][1]` but not taking `max`
- Using `>` for overlap instead of `>=` (off-by-one)
- Modifying the original interval array while iterating
- Not handling empty input

14. Edge Cases

- Empty input → []
- Single interval → [interval]
- All overlapping → [[min_start, max_end]]
- No overlapping → same as input
- Nested intervals [1,10], [2,3] → merged to [1,10]
- Equal intervals [1,5], [1,5] → [1,5]

15. Variations

Variation	What Changes
Insert Interval	Insert a new interval into a sorted list and merge. Can be $O(n)$ without sorting.
Interval List Intersections	Find intersections of two sorted interval lists. Two-pointer.
Interval Union	Same as merge intervals.
Data Stream as Disjoint Intervals	Intervals inserted one at a time. Use ordered set/map.

16. Related Algorithms

Algorithm	Connection
Sweep Line	More general interval processing (overlap counting, skyline)
Non-overlapping Intervals	Inverse: find intervals to remove for no overlap
Meeting Rooms	Detect overlaps (Merge produces non-overlapping result)

17. Practice Problems

Easy

Problem	Platform	Idea
Merge Intervals (LeetCode 56)	LeetCode	Classic
Meeting Rooms (LeetCode 252)	LeetCode	Detect if any overlap exists

Medium

Problem	Platform	Idea
Insert Interval (LeetCode 57)	LeetCode	Insert and merge in one pass
Interval List Intersections (LeetCode 986)	LeetCode	Two-pointer intersection of two lists

Hard

Problem	Platform	Idea
Employee Free Time (LeetCode 759)	LeetCode	Gaps between merged intervals
Data Stream as Disjoint Intervals (LeetCode 352)	LeetCode	Dynamic interval merging with set

18. Interview Explanation

"Merge Intervals is straightforward: sort by start time, then iterate. Maintain a 'current' interval. If the next interval overlaps (its start $\leq$ current end), merge by taking the max end. Otherwise, finalize the current interval and move to the next. Sorting guarantees that overlapping intervals are adjacent. $O(n \log n)$ time, $O(n)$ space for output."

19. Revision Notes

- Sort by **start**
- Overlap: `next.start <= current.end`
- Merge: `current.end = max(current.end, next.end)`
- **$O(n \log n)$** time
- Empty input $\rightarrow$ return `[]`

20. Final Cheat Sheet

Use	Action	Time	Key
Merge overlapping intervals	Sort by start, linear merge	$O(n \log n)$	<code>if(next_start <= last_end) last_end = max(last_end, next_end)</code>

Non-Overlapping Intervals

1. Overview

Given an array of intervals `[start, end]`, find the **minimum number of intervals to remove** so that the remaining intervals are non-overlapping. This is the complement of Activity Selection.

2. Intuition

Simple explanation: The maximum number of non-overlapping intervals you can keep = Activity Selection answer. So minimum to remove = total - maximum keepable.

Analogy: You have too many meetings scheduled. You want to cancel as few as possible so that none overlap. The solution: find the maximum set of non-overlapping meetings (keep them), cancel the rest.

Step-by-step reasoning:

1. Find max non-overlapping intervals (Activity Selection: sort by end, greedy pick)
2. Answer = $n - \text{max_non_overlapping}$

Why it works: This is pure complement of activity selection. The intervals you keep are non-overlapping and maximum in count. Everything else must be removed.

3. When to Use It

- "Minimum intervals to remove to make non-overlapping"
- "Minimum removal to avoid overlap"
- Trigger phrases: "*non-overlapping*", "*remove*", "*erase*", "*overlap*", "*conflict*"

4. When Not to Use It

- When intervals have **weights** (Weighted Interval Scheduling → DP)
- When you need to **merge** rather than remove
- When removal has a per-interval cost (different optimization)

5. Core Concepts

Concept	Explanation	Why It Matters
Complement property	$\text{min_removals} = n - \text{max_non_overlapping}$	Directly connects to Activity Selection
End-time sorting	Sort by end time for max retainable	Same as Activity Selection
Overlap counting	How many intervals a given interval overlaps with	Not directly the answer, but useful context

6. Step-by-Step Algorithm

1. If intervals is empty, return 0
2. Sort intervals by **end time**
3. Initialize **count** = 1 (pick first), **lastEnd** = `intervals[0][1]`
4. For $i = 1$ to $n-1$:
 - If `intervals[i][0] >= lastEnd`: **count++**, **lastEnd** = `intervals[i][1]`
5. Return **n - count**

7. Dry Run

Input: `[[1,2], [2,3], [3,4], [1,3]]`

Sorted by end: `[[1,2], [2,3], [1,3], [3,4]]`

i	Interval	lastEnd	start >= lastEnd?	count
1	[2,3]	2	Yes	2
2	[1,3]	3	No	2
3	[3,4]	3	Yes	3

Max non-overlapping: 3

Min removals: $4 - 3 = 1$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int eraseOverlapIntervals(vector<vector<int>> &intervals) {
    if (intervals.empty()) return 0;
    int n = intervals.size();

    sort(intervals.begin(), intervals.end(),
         [](auto &a, auto &b) { return a[1] < b[1]; });

    int count = 1, lastEnd = intervals[0][1];
    for (int i = 1; i < n; i++) {
        if (intervals[i][0] >= lastEnd) {
            count++;
            lastEnd = intervals[i][1];
        }
    }
    return n - count;
}

int main() {
    vector<vector<int>> intervals = {{1,2}, {2,3}, {3,4}, {1,3}};
    cout << eraseOverlapIntervals(intervals); // 1
    return 0;
}
```

9. Python Implementation

```
def erase_overlap_intervals(intervals):
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1])

    count = 1
    last_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start >= last_end:
            count += 1
            last_end = end

    return len(intervals) - count

intervals = [[1,2], [2,3], [3,4], [1,3]]
print(erase_overlap_intervals(intervals)) # 1
```

10–19. (Non-Overlapping Intervals is the complement of Activity Selection. Code explanation, complexity, patterns, mistakes, edge cases, variations, related algorithms, practice, interview explanation, revision, and cheat sheet follow the same logic as Activity Selection but compute $n - \text{max_non_overlapping}$.)

20. Final Cheat Sheet

Use	Action	Time	Key
Min removals for non-overlap	Find max non-overlapping, subtract from n	$O(n \log n)$	<code>return n - maxNonOverlapping(intervals)</code>

Huffman Coding

1. Overview

Huffman Coding is a **lossless data compression algorithm** that assigns variable-length binary codes to characters based on their frequencies. Frequently occurring characters get shorter codes, less frequent ones get longer codes. It produces a **prefix-free code** (no code is a prefix of another).

2. Intuition

Simple explanation: Count frequency of each character. Build a binary tree where characters are leaves, and the path to each leaf (0 for left, 1 for right) is its code. Merge the two least frequent nodes repeatedly — this ensures frequent characters end up near the root (short codes).

Analogy: In English, 'e' is the most common letter. Morse code uses `.` (short) for 'E'. Huffman coding does the same thing automatically — it gives the shortest codes to the most frequent characters.

Step-by-step reasoning:

1. Count frequencies of all characters
2. Create a leaf node for each character with its frequency
3. Repeatedly take the two nodes with smallest frequency, combine them into a new node (frequency = sum), add back
4. When one node remains, that's the root of the Huffman tree
5. Traverse the tree: left edge = '0', right edge = '1'
6. The path from root to each leaf gives its code

Why it works: The greedy choice is to merge the two smallest frequencies at each step. This minimizes the weighted path length (frequency $\times$ depth) because rare characters end up deeper (doesn't cost much) and frequent ones end up shallower.

3. When to Use It

- "Optimal binary codes for data compression"
- "Minimum weighted path length"

- "Prefix-free code generation"
- Trigger phrases: "*Huffman*", "*encoding*", "*compression*", "*optimal code*", "*prefix code*"

4. When Not to Use It

- When symbol frequencies are **uniform** (Huffman gives no benefit over fixed-length codes)
- When you need **adaptive/streaming** compression (use LZW, gzip)
- When the alphabet is very large (computing tree is expensive)
- When you need **decompression speed** over compression ratio (use fixed tables)

5. Core Concepts

Concept	Explanation	Why It Matters
Prefix-free code	No codeword is a prefix of another	Ensures unambiguous decoding
Frequency	How often a symbol appears	Determines code length
Merge operation	Combine two smallest-frequency nodes	The greedy step
Min-heap / priority queue	Efficient way to repeatedly find two smallest frequencies	Makes algorithm $O(n \log n)$
Weighted path length	Sum of (frequency $\times$ depth) for all symbols	What Huffman minimizes
Huffman tree	Full binary tree with symbols at leaves	The output data structure

6. Step-by-Step Algorithm

1. Count frequency of each character in the input
2. Create a min-heap of nodes (each node: character, frequency, left child, right child)
3. While heap has > 1 node:
 - Extract two nodes with smallest frequency (x, y)
 - Create new node z with frequency = $x.freq + y.freq$, children = x, y
 - Push z back into heap
4. The last remaining node is the root of the Huffman tree
5. Traverse tree (DFS) to generate codes:
 - Left: append '0'
 - Right: append '1'
 - When at a leaf, store the code for that character

7. Dry Run

Input: "aabbcc"

Frequencies: a:2, b:2, c:1

Min-heap: [(1,c), (2,a), (2,b)]

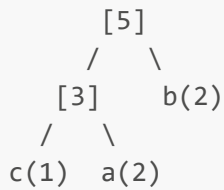
Step 1: Extract c(1) and a(2). Create internal node with freq=3.

Heap: [(2,b), (3,internal)]

Step 2: Extract b(2) and internal(3). Create root with freq=5.

Heap: [(5,root)]

Tree:



Codes:

- c: 00
- a: 01
- b: 1

Encoded string: aabbcc → 01 01 1 1 00 → 01011100

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct Node {
    char ch;
    int freq;
    Node *left, *right;
    Node(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
};

struct Compare {
    bool operator()(Node *a, Node *b) {
        return a->freq > b->freq;
    }
};

// Generate Huffman codes by traversing the tree
void encode(Node *root, string code, unordered_map<char, string> &codes) {
    if (!root) return;
    if (!root->left && !root->right) { // leaf
        codes[root->ch] = code;
        return;
    }
    encode(root->left, code + "0", codes);
    encode(root->right, code + "1", codes);
}

unordered_map<char, string> huffmanCoding(string text) {
    // 1. Count frequencies

```

```

unordered_map<char,int> freq;
for (char c : text) freq[c]++;

// 2. Build min-heap
priority_queue<Node*, vector<Node*>, Compare> pq;
for (auto &p : freq)
    pq.push(new Node(p.first, p.second));

// 3. Build Huffman tree
while (pq.size() > 1) {
    Node *left = pq.top(); pq.pop();
    Node *right = pq.top(); pq.pop();
    Node *internal = new Node('\0', left->freq + right->freq);
    internal->left = left;
    internal->right = right;
    pq.push(internal);
}

// 4. Generate codes
unordered_map<char,string> codes;
encode(pq.top(), "", codes);
return codes;
}

int main() {
    string text = "aabbcc";
    auto codes = huffmanCoding(text);

    cout << "Huffman Codes:\n";
    for (auto &p : codes)
        cout << p.first << " : " << p.second << "\n";

    string encoded = "";
    for (char c : text) encoded += codes[c];
    cout << "Encoded: " << encoded << "\n";
    return 0;
}

```

9. Python Implementation

```

import heapq

class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

```

```

def huffman_coding(text):
    # 1. Count frequencies
    freq = {}
    for c in text:
        freq[c] = freq.get(c, 0) + 1

    # 2. Build min-heap
    heap = [Node(char, f) for char, f in freq.items()]
    heapq.heapify(heap)

    # 3. Build tree
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        internal = Node(None, left.freq + right.freq)
        internal.left = left
        internal.right = right
        heapq.heappush(heap, internal)

    # 4. Generate codes
    codes = {}

    def encode(node, code):
        if not node:
            return
        if not node.left and not node.right:
            codes[node.char] = code
            return
        encode(node.left, code + "0")
        encode(node.right, code + "1")

    encode(heap[0], "")
    return codes

text = "aabbcc"
codes = huffman_coding(text)
print("Huffman Codes:", codes)
encoded = ''.join(codes[c] for c in text)
print("Encoded:", encoded)

```

10. Code Explanation

- **Node struct:** Each tree node has a character (empty for internal nodes), frequency, and left/right children.
- **Min-heap (priority_queue):** Efficiently extracts the two smallest frequencies. The `Compare` functor implements `>` for min-heap behavior (priority_queue is max-heap by default).
- **Tree building loop:** Repeatedly merges smallest two nodes. Each merge creates an internal node with sum frequency.
- **encode function:** DFS traversal. Going left adds '0', right adds '1'. At a leaf, the accumulated code string is saved.

- **Time complexity:** $O(n \log n)$ where n is the number of **unique characters**, not the text length.

11. Complexity Analysis

Operation	Complexity
Frequency counting	$O(N)$ where N = text length
Heap operations (k unique chars)	$O(k \log k)$
Tree construction	$O(k \log k)$
Code generation (DFS)	$O(k)$
Overall	$O(N + k \log k)$
Space	$O(k)$ for tree and codes

Note: $k \leq 256$ (ASCII) or $k \leq 1,114,112$ (Unicode), so for practical purposes k is bounded.

12. Common Patterns

Pattern	How to Identify	Approach
Optimal prefix codes	"Compress", "encode", "minimum bits"	Huffman coding
Minimum weighted path length	"Merge files with costs"	Same as Huffman tree
Greedy on frequencies	"Repeatedly combine smallest"	Min-heap based merging

13. Common Mistakes

- Forgetting that the tree is built from **unique characters**, not text length
- Not handling single character input (tree has only one node)
- Memory leaks in C++ (not deleting nodes)
- Wrong comparison direction for min-heap
- Not handling characters that appear 0 times

14. Edge Cases

- Empty string → no codes
- Single character → code should be "0" or "1" (or empty, but typically "0")
- All characters have same frequency → balanced tree, codes similar length
- Two characters → simple two-node tree
- Very large frequency disparities

15. Variations

Variation	What Changes	When Used
Canonical Huffman	Standardizes code assignment (numerical order)	Required by many compression formats

Variation	What Changes	When Used
Adaptive Huffman	Updates tree dynamically as text streams	Streaming compression
N-ary Huffman	Merge N smallest nodes instead of 2	When output uses N symbols
Huffman for files	Works on blocks or bytes, not just chars	Real compression tools

16. Related Algorithms

Algorithm	Connection
Lossless Compression (LZW, Run-Length)	Different strategies for compression
Min-heap / Priority Queue	Core data structure used in implementation
Binary Tree Traversal	Used to generate codes from the tree
Optimal Merge Pattern	Same algorithmic structure (merge smallest two)

17. Practice Problems

Easy

Problem	Platform	Idea
Huffman Decoding	GFG	Given Huffman tree and encoded string, decode
Minimum Cost of Ropes	GFG	Same greedy merging of smallest two (no tree)

Medium

Problem	Platform	Idea
Huffman Encoding	GFG	Full implementation
Minimum Cost to Connect Sticks (LeetCode 1167)	LeetCode	Same as min cost ropes

Hard

Problem	Platform	Idea
File Compression	Codeforces	Huffman with file blocks
Text Compression Ratio Maximization	Various	Apply Huffman with context

18. Interview Explanation

"Huffman Coding is a greedy algorithm for lossless data compression. It assigns variable-length binary codes to characters based on frequency — more frequent characters get shorter codes. The algorithm repeatedly merges the two lowest-frequency nodes using a min-heap, building a binary tree where leaves represent characters. Codes are generated by tree traversal: left is '0', right is '1'. The greedy

property — always merging the smallest two frequencies — ensures that the resulting tree has minimum weighted path length. Building the tree takes $O(k \log k)$ for k unique characters, and encoding the entire text takes $O(N)$."

19. Revision Notes

- **Basic idea:** Frequent characters → short codes
- **Data structure:** Min-heap (priority queue)
- **Algorithm:** Merge two smallest frequencies until one node remains
- **Codes:** DFS traversal, left = '0', right = '1'
- **Key property:** Prefix-free (no code is prefix of another)
- **Complexity:** $O(N + k \log k)$ where N = text length, k = unique chars
- **Time-tested trick:** Minimum Cost of Ropes uses same "merge smallest two" logic

20. Final Cheat Sheet

Use	Action	Time	Key
Optimal lossless compression	Build tree by merging smallest frequencies	$O(N + k \log k)$	Min-heap of frequency nodes; merge two smallest

Greedy with Heap

1. Overview

Greedy with Heap is a pattern where a **priority queue (heap)** is used to make greedy choices dynamically. Unlike sorting-based greedy where all decisions are made in a fixed order, heap-based greedy can adapt to changing state — the "best" item to process next may change as new information becomes available.

2. Intuition

Simple explanation: At each step, we need the "current best" item. Sorting once doesn't work because the ordering changes as we process items. A heap lets us efficiently retrieve and update the best candidate at each step.

Analogy: You're managing a hospital ER. Patients arrive over time with different severity. You can't sort once and treat in that order because new patients keep arriving. A priority queue (heap) lets you always treat the most critical patient currently waiting.

Step-by-step reasoning:

1. Identify what "best" means (smallest, largest, highest ratio, etc.)
2. Use a min-heap or max-heap to maintain candidates
3. At each step, pop the best from the heap, process it, and possibly push new candidates

Why it works: The heap maintains the invariant that we can always access the optimal next choice in $O(\log n)$ time. This is useful when the optimal choice at step k depends on choices made at steps $1..k-1$.

3. When to Use It

- The "best" item to process changes dynamically
- Items arrive over time or are revealed as others are processed
- Problems with a "combination" or "merge" aspect
- Trigger phrases: "connect", "streaming", "running median", "merge k sorted", "minimum cost to connect"

4. When Not to Use It

- When static sorting suffices ($O(n \log n)$ sort + $O(n)$ scan is simpler)
- When the heap property isn't needed (a simple variable works)
- When the heap overhead exceeds the benefit (small n)
- When $O(\log n)$ per operation is too slow for the constraints

5. Core Concepts

Concept	Explanation	Why It Matters
Min-heap	Parent $\leq$ children; root is smallest	When smallest is best (e.g., merge smallest two)
Max-heap	Parent $\geq$ children; root is largest	When largest is best (e.g., select largest profit first)
Dynamic ordering	The priority of items can change mid-process	Why heap is needed over static sort
Lazy deletion	Mark items as invalid instead of removing from heap	Used when priorities change and we can't easily update
Push/pop cycle	Repeatedly extract best, process, and insert new items	The core loop pattern

6. Step-by-Step Algorithm

1. Initialize a min-heap (or max-heap depending on problem)
2. Insert initial items into heap
3. While condition:
 - Pop the best element from heap
 - Process it
 - Optionally push new elements (derived from processing)
4. Return result

7. Dry Run

Problem: Minimum Cost to Connect Ropes = connect ropes with costs [4, 3, 2, 6]. Cost to connect two ropes = sum of their lengths. Minimize total cost.

Initial heap: [2, 3, 4, 6]

Step	Pop	Pop	Connect	Push (sum)	Total Cost	Heap
1	2	3	2+3=5	5	5	[4, 5, 6]
2	4	5	4+5=9	9	14	[6, 9]
3	6	9	6+9=15	15	29	[15]

Result: Minimum total cost = 29

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Minimum Cost to Connect Ropes
int minCostToConnectRopes(vector<int> &ropes) {
    priority_queue<int, vector<int>, greater<int>> pq(ropes.begin(), ropes.end());

    int totalCost = 0;
    while (pq.size() > 1) {
        int a = pq.top(); pq.pop();
        int b = pq.top(); pq.pop();
        int cost = a + b;
        totalCost += cost;
        pq.push(cost);
    }
    return totalCost;
}

// Example: Find K most frequent elements (using heap)
vector<int> topKFrequent(vector<int> &nums, int k) {
    unordered_map<int,int> freq;
    for (int x : nums) freq[x]++;

    // Min-heap of pairs (frequency, value)
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>>
pq;

    for (auto &p : freq) {
        pq.push({p.second, p.first});
        if (pq.size() > k) pq.pop();
    }

    vector<int> result;
    while (!pq.empty()) {
        result.push_back(pq.top().second);
        pq.pop();
    }
    return result;
}

int main() {
```

```

vector<int> ropes = {4, 3, 2, 6};
cout << "Min cost: " << minCostToConnectRopes(ropes) << "\n"; // 29

vector<int> nums = {1,1,1,2,2,3};
auto topK = topKFrequent(nums, 2);
for (int x : topK) cout << x << " "; // 1 2
return 0;
}

```

9. Python Implementation

```

import heapq

def min_cost_to_connect_ropes(ropes):
    heapq.heapify(ropes)
    total = 0
    while len(ropes) > 1:
        a = heapq.heappop(ropes)
        b = heapq.heappop(ropes)
        cost = a + b
        total += cost
        heapq.heappush(ropes, cost)
    return total

def top_k_frequent(nums, k):
    freq = {}
    for x in nums:
        freq[x] = freq.get(x, 0) + 1

    heap = []
    for val, cnt in freq.items():
        heapq.heappush(heap, (cnt, val))
        if len(heap) > k:
            heapq.heappop(heap)

    return [val for cnt, val in heap]

ropes = [4, 3, 2, 6]
print(min_cost_to_connect_ropes(ropes)) # 29

```

10. Code Explanation

- **Min-heap declaration:** `priority_queue<int, vector<int>, greater<int>>` — `greater<int>` makes it a min-heap.
- **While `size > 1`:** We need at least two ropes to connect. Last remaining rope is the final connected rope.
- **Push back sum:** The combined rope becomes a new candidate for future connections.
- **Top K Frequent:** Min-heap of size k. If heap exceeds k, pop the smallest frequency. At the end, the k largest frequencies remain.

11. Complexity Analysis

Operation	Complexity
Building heap	$O(n)$
Each pop/push	$O(\log n)$
Overall (n operations)	$O(n \log n)$
Space	$O(n)$

12. Common Patterns

Pattern	How to Identify	Approach
Merge smallest two	"Connect ropes", "merge files"	Min-heap, pop two, push sum
Top K / K smallest/largest	"K most frequent", "K closest"	Min-heap of size k for top k
Median in stream	Running median of stream	Two heaps (max-heap for lower half, min-heap for upper half)
Dijkstra	Shortest path	Min-heap of distances
Prim's MST	Minimum spanning tree	Min-heap of edge weights

13. Common Mistakes

- Using default `priority_queue` (max-heap) when min-heap is needed
- Forgetting `greater<int>` for min-heap in C++
- Not handling empty heap pop
- Pushing then popping without checking size
- Using $O(n)$ linear search when heap gives $O(\log n)$

14. Edge Cases

- Single element $\rightarrow$ 0 cost (no connection needed)
- All equal elements
- Very large values (possible overflow in sum)
- Empty input
- $k >$ number of unique elements (return all)

15. Variations

Variation	What Changes
Dual heap	Two heaps for median, or one for Dijkstra relaxation
Lazy deletion heap	When priorities change, mark stale and skip on pop

Variation	What Changes
Indexed priority queue	Efficiently update key of specific element (decrease-key)
Bounded heap	Fixed-size heap for top-k problems

16. Related Algorithms

Algorithm	Connection
Sorting	Alternative for static problems. Heap for dynamic.
Balanced BST	Also provides min/max but with $O(\log n)$ insert/delete
Segment Tree	More powerful but more complex. Heap for simpler ordering needs.

17. Practice Problems

Easy

Problem	Platform	Idea
Kth Largest Element in a Stream (LeetCode 703)	LeetCode	Min-heap of size k
Minimum Cost of Ropes	GFG	Classic heap merge

Medium

Problem	Platform	Idea
Top K Frequent Elements (LeetCode 347)	LeetCode	Freq map + min-heap
Find Median from Data Stream (LeetCode 295)	LeetCode	Two heaps
Course Schedule III (LeetCode 630)	LeetCode	Sort by deadline + max-heap

Hard

Problem	Platform	Idea
Minimum Cost to Hire K Workers (LeetCode 857)	LeetCode	Sort by ratio + min-heap
IPO (LeetCode 502)	LeetCode	Two heaps: capital sorted + profit max-heap
Sliding Window Median (LeetCode 480)	LeetCode	Two heaps with lazy deletion

18. Interview Explanation

"Greedy with heap is used when the optimal candidate at each step isn't known in advance but depends on previous choices. The heap ensures $O(\log n)$ access to the best candidate. Classic examples: Dijkstra's shortest path uses a min-heap of distances; connecting ropes always merges the

two smallest via a min-heap; top-k frequent elements uses a min-heap of size k. The pattern is: initialize heap, repeatedly pop the best, process it, and push new candidates back."

19. Revision Notes

- **Use heap when:** ordering changes dynamically
- **Min-heap:** `priority_queue<int, vector<int>, greater<int>>`
- **Max-heap:** `priority_queue<int>` (default)
- **Top-k pattern:** Keep heap size = k; push then pop if size > k
- **Merge pattern:** Pop two smallest, push sum
- **Complexity:** $O(n \log n)$ for n heap operations

20. Final Cheat Sheet

Use	Heap Type	Complexity	Key
Always pick smallest two	Min-heap	$O(n \log n)$	pop two → sum → push
Keep top k largest	Min-heap of size k	$O(n \log k)$	push then if size>k pop
Keep top k smallest	Max-heap of size k	$O(n \log k)$	Same idea, max-heap
Running median	Two heaps	$O(\log n)$ per element	Left: max-heap, Right: min-heap

Greedy with Stack

1. Overview

Greedy with Stack is a pattern where a **stack** is used to build a result by making local decisions that may later be reverted. It's common in problems where we need to remove elements under certain constraints, or build the lexicographically smallest/largest result.

2. Intuition

Simple explanation: As we process elements, we push them onto a stack. When a new element arrives, we check if popping existing elements leads to a better result. The stack maintains the "best solution so far" and the greedy condition determines when to backtrack.

Analogy: Building a tower with blocks. You place blocks one by one. If a new block is better than one already placed (e.g., larger/more suitable), you can remove the old one and put the new one instead. The stack always has the current optimal construction.

Step-by-step reasoning:

1. Process elements left to right
2. At each element, check if the top of stack should be removed (based on some greedy criterion)
3. While condition holds, pop from stack
4. Push current element
5. The stack contains the optimal selection at any point

Why it works: The stack's LIFO nature lets us undo previous decisions in reverse order. This is useful when the optimality of a previous decision depends on future elements.

3. When to Use It

- "Remove K digits to make smallest number"
- "Remove duplicate letters to make lexicographically smallest"
- Building minimal/maximal sequences with removal constraints
- Trigger phrases: "remove", "lexicographically smallest", "largest", "stack", "monotonic"

4. When Not to Use It

- When removals are arbitrary (not constrained to adjacency or order)
- When the problem is about **selecting** elements rather than removing them (use different approach)
- When you only need a simple pass without backtracking (use simple greedy)

5. Core Concepts

Concept	Explanation	Why It Matters
Monotonic stack	Stack elements are in monotonic (increasing/decreasing) order	Core pattern; greedy maintained ordering
Removal condition	When to pop: usually when top > current AND we have removals left	Decides when backtracking happens
Limited removals	Constraint on how many elements can be removed	Makes problem well-defined; greedy must respect budget
LIFO backtracking	Most recently added element is the first to be removed	Necessitated by the nature of the domain

6. Step-by-Step Algorithm

Example: Remove K Digits to get smallest number

1. Initialize empty stack
2. For each digit in the number string:
 - While stack not empty AND top > current digit AND k > 0:
 - Pop stack, decrement k
 - Push current digit
3. If k > 0 (removals remaining), pop from end (remove largest remaining digits)
4. Build result from stack, remove leading zeros

7. Dry Run

Problem: Remove K=3 digits from "1432219" to get smallest number.

Digit	Stack Before	Condition	Action	Stack After
1	[]	-	Push	[1]

Digit	Stack Before	Condition	Action	Stack After
4	[1]	4 > 1? No	Push	[1,4]
3	[1,4]	4 > 3? Yes, k=3	Pop 4, k=2	[1]
3	[1]	1 > 3? No	Push	[1,3]
2	[1,3]	3 > 2? Yes, k=2	Pop 3, k=1	[1]
2	[1]	1 > 2? No	Push	[1,2]
2	[1,2]	2 > 2? No	Push	[1,2,2]
1	[1,2,2]	2 > 1? Yes, k=1	Pop 2, k=0	[1,2]
1	[1,2]	k=0	Push	[1,2,1]
9	[1,2,1]	k=0	Push	[1,2,1,9]

k=0, so no more removals. Stack: [1,2,1,9] → Result: "1219"

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Remove K digits to make smallest number
string removeKdigits(string num, int k) {
    if (k >= num.size()) return "0";

    string stack;
    for (char digit : num) {
        while (!stack.empty() && stack.back() > digit && k > 0) {
            stack.pop_back();
            k--;
        }
        stack.push_back(digit);
    }

    // If k > 0, remove from end (largest remaining)
    while (k > 0) {
        stack.pop_back();
        k--;
    }

    // Remove leading zeros
    int i = 0;
    while (i < stack.size() && stack[i] == '0') i++;
    string result = stack.substr(i);
    return result.empty() ? "0" : result;
}

int main() {
```

```

cout << removeKdigits("1432219", 3) << "\n"; // "1219"
cout << removeKdigits("10200", 1) << "\n"; // "200"
cout << removeKdigits("10", 2) << "\n"; // "0"
return 0;
}

```

9. Python Implementation

```

def remove_k_digits(num: str, k: int) -> str:
    if k >= len(num):
        return "0"

    stack = []
    for digit in num:
        while stack and stack[-1] > digit and k > 0:
            stack.pop()
            k -= 1
        stack.append(digit)

    # If k > 0, remove from end
    while k > 0:
        stack.pop()
        k -= 1

    # Remove leading zeros
    result = ''.join(stack).lstrip('0')
    return result if result else "0"

print(remove_k_digits("1432219", 3)) # "1219"
print(remove_k_digits("10200", 1))   # "200"
print(remove_k_digits("10", 2))      # "0"

```

10. Code Explanation

- **Stack as string:** Using `string` as a stack for easy access to back. `stack.back()` is the top.
- **while (!stack.empty() && stack.back() > digit && k > 0):** Greedy condition. If the previous digit is larger than current digit, removing it makes the number smaller. We have a limited budget `k` for removals.
- **Post-loop removals:** If `k > 0` after processing all digits, we remove from the end (the rightmost digits are the largest remaining — removing them minimizes the number).
- **Leading zero removal:** The result string can't start with '0' unless it's just "0".
- **string as stack vs stack<char>:** `string` is simpler because we need to build the result from it.

11. Complexity Analysis

Operation	Complexity
Single pass	O(n)

Operation	Complexity
Each digit pushed once, popped at most once	$O(n)$ amortized
Overall	$O(n)$
Space	$O(n)$

12. Common Patterns

Pattern	How to Identify	Approach
Remove K digits	"Remove k elements to make smallest/largest number"	Monotonic increasing stack
Remove duplicate letters	"Lexicographically smallest by removing duplicates"	Stack with frequency count and visited set
Build largest number	"Form largest number from digits with removals"	Monotonic decreasing stack
Parse with backtracking	"Remove patterns from string"	Stack, check condition on top

13. Common Mistakes

- Forgetting to handle remaining removals after the loop
- Not removing leading zeros
- Using `k >= n` early return but forgetting it
- Using `>=` instead of `>` (or vice versa) for the greedy comparison
- Not using `while` (using `if` instead) — only pop one, but may need multiple pops

14. Edge Cases

- `k >= n` → "0"
- `k = 0` → original number
- All digits same → just remove last k digits
- Number with leading zeros after removal → strip them
- "0" → "0" (even with `k=0`)
- Decreasing digits → must use removal budget
- Increasing digits → no pops, removals from end

15. Variations

Variation	What Changes
Largest number after K removals	Reverse comparison: <code>while top < digit</code>
Remove duplicate letters	<code>stack.back() > current</code> AND <code>current</code> still appears later

Variation	What Changes
Build smallest sequence with constraints	Similar stack pattern with additional constraints
Monotonic stack (general)	General pattern for next greater/smaller element

16. Related Algorithms

Algorithm	Connection
Monotonic Stack	The foundation; used for next greater/smaller element in $O(n)$
Greedy on strings	General string optimization with greedy choices
Two-pointer greedy	Alternative for some removal problems

17. Practice Problems

Easy

Problem	Platform	Idea
Remove Outermost Parentheses (LeetCode 1021)	LeetCode	Stack + count
Minimum Add to Make Parentheses Valid (LeetCode 921)	LeetCode	Stack for unmatched

Medium

Problem	Platform	Idea
Remove K Digits (LeetCode 402)	LeetCode	Classic monotonic stack
Remove Duplicate Letters (LeetCode 316)	LeetCode	Lexicographically smallest subsequence
Smallest Subsequence of Distinct Characters (LeetCode 1081)	LeetCode	Same as 316

Hard

Problem	Platform	Idea
Remove Invalid Parentheses (LeetCode 301)	LeetCode	BFS/DFS, not purely stack greedy
Create Maximum Number (LeetCode 321)	LeetCode	Combine from two arrays with monotonic stack

18. Interview Explanation

"Greedy with stack is useful when we need to remove elements to optimize a sequence, like making the smallest possible number after removing K digits. We iterate left to right. At each digit, we pop larger digits from the stack while we have removals remaining — this makes the number smaller because replacing a larger digit with a smaller one earlier produces a smaller number. After the pass, if removals

remain, we remove from the end. This is $O(n)$ because each element is pushed and popped at most once."

19. Revision Notes

- **Pattern:** Process left to right, remove "worse" earlier elements
- **Stack:** `string` works as stack for building result
- **Condition:** Usually `while top > current && k > 0`
- **Remaining removals:** Handle after loop (remove from end)
- **Leading zeros:** Strip them
- **$O(n)$ time, $O(n)$ space**

20. Final Cheat Sheet

Use	Stack Order	Condition	Time	Key
Remove K digits (smallest)	Monotonic increasing	<code>top > digit && k > 0</code>	$O(n)$	pop larger digits, push current
Remove K digits (largest)	Monotonic decreasing	<code>top < digit && k > 0</code>	$O(n)$	pop smaller digits, push current
Remove duplicate letters	Increasing with frequency check	<code>top > curr && freq[top] > 0</code>	$O(n)$	Track last occurrence and visited set

Greedy on Graphs

1. Overview

Greedy on Graphs refers to graph algorithms that make locally optimal choices at each step to solve problems like shortest paths (Dijkstra), minimum spanning tree (Prim, Kruskal), and maximum bipartite matching. These algorithms are optimal because the underlying problems satisfy the **greedy choice property**.

2. Intuition

Simple explanation: In graph problems, greedy algorithms always pick the "best" next vertex or edge (cheapest edge, nearest vertex) without reconsidering. For certain problems (shortest paths with non-negative weights, MST), this local choice never hurts the global solution.

Analogy (MST): You're building a road network connecting cities. You always build the cheapest road that connects a new city to the already-connected set. This gives the cheapest network overall.

Analogy (Dijkstra): You're navigating a map. You always expand to the nearest unvisited city from your known region. The first time you reach a city, you've found the shortest path to it.

Why it works: These graph problems have **optimal substructure** (the best path to B through A contains the best path to A) and the **greedy choice property** (locally best edges/vertices are part of some globally optimal solution).

3. When to Use It

- **Shortest paths** in non-negative weighted graphs → Dijkstra
- **Minimum spanning tree** → Prim (vertex-based) or Kruskal (edge-based)
- **Maximum bipartite matching** → Greedy matching in certain cases
- **Topological ordering** → Kahn's algorithm (DAG greedy)
- Trigger phrases: "*shortest path*", "*MST*", "*minimum spanning*", "*Dijkstra*", "*Prim*", "*Kruskal*"

4. When Not to Use It

- Graphs with **negative edge weights** (Dijkstra fails; use Bellman-Ford)
- **Longest path** in general graph (NP-hard; DP on DAG works)
- **Traveling Salesman Problem** (need approximation algorithms)
- **Maximum flow** (Ford-Fulkerson / Dinic needed)
- **All-pairs shortest paths** (Floyd-Warshall may be better for dense graphs)

5. Core Concepts

Concept	Explanation	Why It Matters
Relaxation	Updating a tentative distance with a shorter path	Core operation in shortest path algorithms
Cut property	For any cut, the minimum weight edge crossing the cut is in some MST	What makes Prim/Kruskal work
Cycle property	The maximum weight edge in any cycle is NOT in any MST	Kruskal uses this
Priority queue (heap)	Efficiently get "closest" vertex or "cheapest" edge	Core data structure for Dijkstra and Prim
Vertex set / edge set	Track visited vertices (Dijkstra/Prim) or connected components (Kruskal)	Algorithm state

6. Step-by-Step Algorithm

Dijkstra (Single Source Shortest Path)

1. Set $\text{distance}[\text{source}] = 0$, all others $= \infty$
2. Push $(0, \text{source})$ into min-heap
3. While heap not empty:
 - Pop (dist, u) . If $\text{dist} > \text{distance}[u]$, continue (stale entry)
 - For each neighbor v with edge weight w :
 - If $\text{distance}[u] + w < \text{distance}[v]$: relax → update $\text{distance}[v]$, push $(\text{distance}[v], v)$

Prim (MST — vertex based)

1. Start from arbitrary vertex, mark visited
2. Push all edges from source into min-heap
3. While heap not empty and MST incomplete:

- Pop smallest edge (u, v, w). If v visited, skip.
- Add v to visited set, add edge to MST
- Push all edges from v to unvisited neighbors

Kruskal (MST — edge based)

1. Sort all edges by weight
2. Initialize DSU (Union-Find) for all vertices
3. For each edge (u, v, w) in sorted order:
 - If $\text{find}(u) \neq \text{find}(v)$: add edge to MST, $\text{union}(u, v)$
4. Stop when MST has $n-1$ edges

7. Dry Run

Graph: 4 vertices, edges: (0-1:10), (0-2:3), (1-2:1), (1-3:2), (2-3:8)

Dijkstra from vertex 0

Step	Pop	Distance array	Processed
Init	-	[0, ∞ , ∞ , ∞]	{}
1	(0,0)	[0, $10 \rightarrow 0+3=3$, 3, ∞]	{0} $\rightarrow$ relax 2: $0+3=3$, 1: $0+10=10$
2	(3,2)	[0, $\min(10, 3+1=4)=4$, 3, $\min(\infty, 3+8=11)=11$]	{0,2} $\rightarrow$ relax 1: $3+1=4$, 3: $3+8=11$
3	(4,1)	[0, 4, 3, $\min(11, 4+2=6)=6$]	{0,2,1} $\rightarrow$ relax 3: $4+2=6$
4	(6,3)	[0, 4, 3, 6]	{0,2,1,3}

Distances: [0, 4, 3, 6]

Kruskal MST

Sorted edges: (1-2:1), (1-3:2), (0-2:3), (2-3:8), (0-1:10)

Edge	$\text{find}(u) \neq \text{find}(v)$?	Action	MST edges	Components
(1-2:1)	$1 \neq 2$	Add	{{(1,2)}}	{0}, {1,2}, {3}
(1-3:2)	$\text{find}(1) \neq \text{find}(3)$	Add	{{(1,2),(1,3)}}	{0}, {1,2,3}
(0-2:3)	$\text{find}(0) \neq \text{find}(1,2,3)$	Add	{{(1,2),(1,3),(0,2)}}	{0,1,2,3} (n-1=3 edges done)

MST total weight: $1+2+3=6$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

typedef pair<int,int> pii;
```

```

// Dijkstra
vector<int> dijkstra(vector<vector<pii>> &graph, int src) {
    int n = graph.size();
    vector<int> dist(n, INT_MAX);
    dist[src] = 0;

    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // stale entry

        for (auto &[v, w] : graph[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

// DSU for Kruskal
struct DSU {
    vector<int> parent, rank;
    DSU(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    void unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return;
        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[px] > rank[py]) parent[py] = px;
        else { parent[py] = px; rank[px]++; }
    }
};

// Kruskal MST
int kruskalMST(int n, vector<vector<int>> &edges) {
    // edges: [u, v, weight]
    sort(edges.begin(), edges.end(), [](auto &a, auto &b) {
        return a[2] < b[2];
    });

    DSU dsu(n);
    int mstWeight = 0, edgesUsed = 0;

```

```

for (auto &e : edges) {
    if (dsu.find(e[0]) != dsu.find(e[1])) {
        dsu.unite(e[0], e[1]);
        mstWeight += e[2];
        edgesUsed++;
        if (edgesUsed == n - 1) break;
    }
}
return mstWeight; // -1 if graph disconnected
}

int main() {
    // Dijkstra
    int n = 4;
    vector<vector<pii>> graph(n);
    graph[0] = {{1,10}, {2,3}};
    graph[1] = {{2,1}, {3,2}};
    graph[2] = {{3,8}};
    graph[3] = {};

    auto dist = dijkstra(graph, 0);
    for (int i = 0; i < n; i++)
        cout << "Dist to " << i << ": " << dist[i] << "\n";

    // Kruskal
    vector<vector<int>> edges = {{0,1,10}, {0,2,3}, {1,2,1}, {1,3,2}, {2,3,8}};
    cout << "MST weight: " << kruskalMST(4, edges) << "\n"; // 6
    return 0;
}

```

9. Python Implementation

```

import heapq

def dijkstra(graph, src):
    n = len(graph)
    INF = float('inf')
    dist = [INF] * n
    dist[src] = 0
    pq = [(0, src)]

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist

```

```

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def unite(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return
        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px
        else:
            self.parent[py] = px
            self.rank[px] += 1

def kruskal_mst(n, edges):
    edges.sort(key=lambda x: x[2]) # sort by weight
    dsu = DSU(n)
    mst_weight = 0
    edges_used = 0

    for u, v, w in edges:
        if dsu.find(u) != dsu.find(v):
            dsu.unite(u, v)
            mst_weight += w
            edges_used += 1
            if edges_used == n - 1:
                break

    return mst_weight if edges_used == n - 1 else -1

# Dijkstra
graph = [[] for _ in range(4)]
graph[0] = [(1,10), (2,3)]
graph[1] = [(2,1), (3,2)]
graph[2] = [(3,8)]
print(dijkstra(graph, 0)) # [0, 4, 3, 6]

# Kruskal
edges = [[0,1,10], [0,2,3], [1,2,1], [1,3,2], [2,3,8]]
print(kruskal_mst(4, edges)) # 6

```

10. Code Explanation

- **Dijkstra:** Min-heap stores (distance, vertex). The `if (d > dist[u]) continue` check skips stale entries (old distances for a vertex that was already updated).
- **Relaxation:** `dist[u] + w < dist[v]` means we found a shorter path to v. Update and push.
- **Kruskal:** Sort edges by weight. DSU tracks connectivity. If an edge connects two different components, adding it won't create a cycle.
- **DSU path compression + union by rank:** Keeps DSU operations nearly $O(1)$.

11. Complexity Analysis

Algorithm	Time	Space
Dijkstra (binary heap)	$O((V+E) \log V)$	$O(V)$
Prim (binary heap)	$O((V+E) \log V)$	$O(V)$
Kruskal (sort + DSU)	$O(E \log E + E \alpha(V))$	$O(V)$

12–19. (Refer to general graph theory resources for detailed patterns, mistakes, edge cases, and practice. The key takeaway: Dijkstra, Prim, and Kruskal are the three classic greedy graph algorithms.)

20. Final Cheat Sheet

Algorithm	Use	Greedy Choice	Time	Data Structure
Dijkstra	Shortest path (non-negative)	Closest unvisited vertex	$O((V+E) \log V)$	Min-heap
Prim	MST (vertex-focused)	Cheapest edge connecting visited $\leftrightarrow$ unvisited	$O((V+E) \log V)$	Min-heap
Kruskal	MST (edge-focused)	Cheapest edge that doesn't form cycle	$O(E \log E)$	Sort + DSU

Greedy Proof Technique

1. Overview

Greedy algorithms are often intuitive but can be wrong. A **proof of correctness** for a greedy algorithm is essential. The two main techniques are:

1. **Exchange Argument (Swapping):** Show that any optimal solution can be transformed into the greedy solution without losing optimality.
2. **Greedy Stays Ahead:** Show that the greedy solution is at least as good as any other solution at every partial step.

2. Intuition

Simple explanation: We need to convince ourselves (and interviewers) that making the locally optimal choice at each step leads to the globally optimal solution. The exchange argument does this by showing that if an optimal solution differs from the greedy one, we can "swap" the difference and still have an optimal solution.

Analogy (Exchange argument): You have a pile of coins and want to make a certain amount using the fewest coins. If the optimal solution uses a different first coin than the greedy (largest-first) approach, you could swap coins and still have a valid solution of the same size. Repeating these swaps eventually transforms the optimal into the greedy.

Why these techniques work: They leverage the two properties required for greedy correctness:

- **Greedy choice property:** A global optimum can be reached by making a local optimal choice
- **Optimal substructure:** An optimal solution contains optimal solutions to subproblems

3. When to Use It

- Proving a greedy algorithm is correct (or finding it's wrong)
- Interview questions: "Prove that this greedy works"
- Before implementing a greedy solution, to ensure it's correct
- After developing a greedy solution that passed sample tests but might fail edge cases

4. When Not to Use It

- The problem has overlapping subproblems without greedy choice property (use DP)
- Trying to prove a greedy algorithm that is actually incorrect
- When the problem is well-known (just cite known proof)

5. Core Concepts

Concept	Explanation	Why It Matters
Greedy choice property	A local optimum leads to a global optimum	The defining property that must be proved
Optimal substructure	The optimal solution to a problem contains optimal solutions to subproblems	Allows recursive/iterative construction
Exchange argument	Take any optimal solution, find first point where it differs from greedy, swap to align	Most common proof technique
Greedy stays ahead	Show that after k steps, the greedy solution is at least as good as any other solution	Alternative to exchange argument
Inductive proof	Use induction to show the invariant holds at each step	Structured way to present the proof

6. Step-by-Step Exchange Argument

1. **Let OPT** be an optimal solution (not necessarily greedy)
2. **Let GREEDY** be the greedy solution
3. **Find first difference** between OPT and GREEDY

4. **Show we can swap** the differing element in OPT with the greedy element without making OPT worse
5. **Reduce** the problem to a smaller instance
6. **By induction**, repeated swaps transform OPT into GREEDY, proving GREEDY is optimal

7. Dry Run (Proof Example)

Theorem: For Activity Selection (sort by finish time, pick earliest non-conflicting), the greedy algorithm gives the maximum number of activities.

Proof by exchange argument:

Let GREEDY = $\{g_1, g_2, \dots, g_k\}$ (activities selected by greedy).

Let OPT = $\{o_1, o_2, \dots, o_m\}$ (any optimal solution, sorted by finish time).

Claim: $k = m$ (greedy is optimal).

Base: g_1 finishes at or before o_1 (greedy picks the earliest-finishing activity, so $g_1.\text{end} \leq o_1.\text{end}$).

Swap: If $g_1 \neq o_1$, replace o_1 with g_1 in OPT. Since g_1 ends no later than o_1 , all other activities in OPT remain non-conflicting. The new set $\{g_1, o_2, \dots, o_m\}$ is still optimal.

Inductive step: After aligning first i activities, consider the subproblem starting after g_i ends. Greedy picks g_{i+1} as the earliest-finishing activity in this subproblem. The same exchange argument shows we can align o_{i+1} with g_{i+1} .

Conclusion: By induction, $k = m$ and greedy is optimal.

8. C++ / 9. Python (Proof Demonstration)

Not applicable — this is a proof technique, not an algorithm. The code examples for greedy algorithms are shown in their respective sections.

10. Code Explanation

Not applicable.

11. Complexity Analysis

Not applicable (this is about proof technique, not an algorithm).

12. Common Patterns

Proof Pattern	When to Use	Example
Exchange argument	Problem has ordering/selection decisions	Activity Selection, Fractional Knapsack
Greedy stays ahead	Problem where greedy produces a sequence, and we compare partial solutions	Scheduling tasks on machines
Inductive proof	Any greedy algorithm	Most greedy proofs use induction

Proof Pattern	When to Use	Example
Matroid properties	Problems with matroid structure (see Matroid Greedy section)	Maximum weight independent set in a matroid

13. Common Mistakes

- Assuming greedy works without proof
- Incomplete exchange argument (not showing all cases)
- Not handling ties correctly in the proof
- Proving optimal substructure but not greedy choice property (both are required)
- Confusing "correct for sample tests" with "mathematically correct"

14. Edge Cases in Proof

- Ties: What if multiple elements have equal greedy value?
- Empty input: Base case for induction
- What if the greedy choice is not unique?
- The "first difference" may not exist (greedy = optimal)

15. Variations

Variation	What Changes
Charge argument	Charge cost of each greedy step to some element, showing total bound
Potential function	Define a function that decreases monotonically with greedy steps
Matroid proof	Use matroid properties for unified proof

16. Related Concepts

Concept	Connection
Dynamic Programming	Proving greedy works = showing DP's redundant subproblems never need revisiting
Matroids	A matroid is precisely a structure where the greedy algorithm works for maximum weight independent set
Exchange Argument in Graphs	Used to prove correctness of Kruskal and Prim

17. Practice

Use the proof techniques on these problems:

Problem	Platform	Proof Technique
Activity Selection	Classic	Exchange argument

Problem	Platform	Proof Technique
Fractional Knapsack	Classic	Exchange argument
Huffman Coding	Classic	Exchange argument (or greedy stays ahead)
Job Sequencing with Deadlines	GFG	Exchange argument
Kruskal's MST	Classic	Cut property + exchange argument

18. Interview Explanation

"To prove a greedy algorithm correct, I use an exchange argument. First, I note the greedy choice — what the algorithm picks at each step. Then I consider any optimal solution OPT. If OPT differs from the greedy solution, I find the first point of difference and show I can swap the greedy element into OPT without making OPT worse. By repeating this, OPT becomes identical to the greedy solution, proving the greedy solution is optimal. A simpler variant is 'greedy stays ahead', where I show that after k steps, the greedy partial solution is at least as good as any other partial solution."

19. Revision Notes

- **Two properties required:** Greedy choice property + optimal substructure
- **Exchange argument:** Find first difference, swap, show no loss
- **Greedy stays ahead:** Show at each step greedy $\geq$ optimal for same prefix
- **Induction:** Used in both techniques
- **Remember:** If you can't prove it, it might be wrong!

20. Final Cheat Sheet

Technique	Idea	Format
Exchange argument	Take optimal, swap differences with greedy	Induction on steps
Greedy stays ahead	Compare partial solutions	Induction on prefix
Key question	Can local choice ever hurt global?	Test with counterexample

Matroid Greedy

1. Overview

A **matroid** is a mathematical structure that generalizes linear independence in vector spaces to abstract sets. The key result: **The greedy algorithm finds the maximum-weight independent set for any matroid.** If a problem can be modeled as finding a maximum-weight independent set in a matroid, the greedy algorithm is guaranteed to be correct.

2. Intuition

Simple explanation: A matroid defines what a "valid" (independent) set looks like. If a problem satisfies the matroid properties, the greedy algorithm of "always pick the highest-value element that keeps the set

independent" gives the optimal solution.

Analogy: Think of a matroid like legal combinations of Legos. Some sets of Legos "fit together legally" (independent), others don't. The greedy algorithm says: take the most valuable Lego piece that still fits with what you already have. If the "fitting" rule is a matroid, this greedy gives the most valuable legal set.

Matroid axioms (simplified):

1. Empty set is independent
2. Subsets of independent sets are independent (hereditary property)
3. If A and B are independent and $|A| < |B|$, there exists $x \in B \setminus A$ such that $A \cup \{x\}$ is independent (augmentation property)

Why it works: The augmentation property ensures that the greedy choice of the maximum-weight element can always be extended to a maximum-weight basis.

3. When to Use It

- Proving greedy is optimal for structures that "look like" linear independence
- Problem has a notion of "independence" with the augmentation property
- Finding maximum spanning tree (graphic matroid)
- Scheduling with matroid constraints
- Trigger phrases: "*matroid*", "*independence*", "*exchange property*", "*greedy works because*"

4. When Not to Use It

- The problem doesn't satisfy augmentation property
- The structure is a greedoid (weaker form) rather than matroid
- You just need the greedy algorithm without the theory (use exchange argument directly)

5. Core Concepts

Concept	Explanation	Why It Matters
Ground set	The set of all elements	The universe from which we select
Independent set	A subset that satisfies the problem's feasibility constraint	The "valid" solutions
Basis	A maximal independent set	Cannot add more elements while maintaining independence
Rank	Size of a maximal independent set	Generalization of dimension
Graphic matroid	Edges of a graph; independent = no cycles	Kruskal's algorithm is the matroid greedy
Uniform matroid	Independent = size $\leq k$ for some k	Selecting up to k items
Transversal matroid	Matchings in bipartite graph	Generalizes assignment problems

6. Step-by-Step Algorithm (Maximum Weight Independent Set in Matroid)

1. Sort all elements by weight descending
2. Initialize independent set $S = \emptyset$
3. For each element e in sorted order:
 - If $S \cup \{e\}$ is independent:
 - Add e to S
4. Return S

This is exactly Kruskal's algorithm when applied to a graphic matroid!

7. Dry Run (Uniform Matroid)

Ground set: {a:10, b:8, c:5, d:3, e:1}

Constraint: Independent = size ≤ 3

Sorted by weight: a(10), b(8), c(5), d(3), e(1)

Element	S before	$S \cup \{e\}$ independent?	Action	S after
a(10)	{}	Yes (size $1 \leq 3$)	Add	{a}
b(8)	{a}	Yes (size $2 \leq 3$)	Add	{a, b}
c(5)	{a,b}	Yes (size $3 \leq 3$)	Add	{a, b, c}
d(3)	{a,b,c}	No (size would be $4 > 3$)	Skip	{a, b, c}
e(1)	{a,b,c}	No	Skip	{a, b, c}

Result: {a, b, c} with total weight 23

8. C++ Implementation (Graphic Matroid — Kruskal)

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, rank;
    DSU(int n) : parent(n), rank(n, 0) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    bool unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[px] > rank[py]) parent[py] = px;
    }
};
```

```

        else { parent[py] = px; rank[px]++; }
        return true;
    }
};

int maxWeightSpanningTree(int n, vector<vector<int>> &edges) {
    // Graphic matroid: independent = acyclic set of edges
    // Max weight → sort descending
    sort(edges.begin(), edges.end(),
        [](auto &a, auto &b) { return a[2] > b[2]; });

    DSU dsu(n);
    int totalWeight = 0, edgesUsed = 0;

    for (auto &e : edges) {
        if (dsu.unite(e[0], e[1])) {
            totalWeight += e[2];
            edgesUsed++;
            if (edgesUsed == n - 1) break;
        }
    }
    return totalWeight;
}

int main() {
    int n = 4;
    vector<vector<int>> edges = {{0,1,10}, {0,2,3}, {1,2,1}, {1,3,2}, {2,3,8}};
    cout << "Max weight spanning tree: " << maxWeightSpanningTree(n, edges);
    return 0;
}

```

9. Python Implementation

```

class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def unite(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False
        if self.rank[px] < self.rank[py]:
            self.parent[px] = py
        elif self.rank[px] > self.rank[py]:
            self.parent[py] = px

```

```

        else:
            self.parent[py] = px
            self.rank[px] += 1
        return True

def max_weight_spanning_tree(n, edges):
    # edges: [u, v, weight]
    # Sort by weight descending
    edges.sort(key=lambda x: -x[2])

    dsu = DSU(n)
    total = 0
    used = 0

    for u, v, w in edges:
        if dsu.unite(u, v):
            total += w
            used += 1
            if used == n - 1:
                break

    return total if used == n - 1 else -1

n = 4
edges = [[0,1,10], [0,2,3], [1,2,1], [1,3,2], [2,3,8]]
print(max_weight_spanning_tree(n, edges)) # 10+8+2=20?
# Sorted: [10,8,3,2,1] → pick 10, stop cycles on 8...
# Let's trace: (0,1,10) ok, (1,3,8) ok, (2,3,3) ok → 10+8+3=21

```

10. Code Explanation

- **Matroid greedy template:** Sort elements by weight descending, check independence before adding.
- **Graphic matroid** → **DSU:** Independence = acyclic. DSU's `find` checks if adding edge creates a cycle.
- **Max vs Min spanning tree:** Matroid greedy works for both. Sort descending for max, ascending for min.

11. Complexity Analysis

Operation	Complexity
Sorting	$O(n \log n)$
Independence checks	$O(n \times \text{check_cost})$
Overall	Depends on independence oracle

For graphic matroid (Kruskal): $O(E \log E)$

12–19. (Matroid theory is broad; the key takeaway for placements/CP is that if you can model a problem as finding maximum weight independent set in a matroid, the greedy algorithm is provably optimal.

Common matroids encountered in CP: graphic matroid (MST), uniform matroid (select k items), partition matroid (resource allocation).

20. Final Cheat Sheet

Matroid Type	Ground Set	Independence	Example Algorithm
Graphic	Edges of graph	Acyclic set of edges	Kruskal's MST
Uniform	Any set	Size $\leq k$	Pick top k by weight
Partition	Elements partitioned into groups	At most k_i from group i	Resource allocation
Transversal	Bipartite graph edges	Matching in bipartite graph	Maximum matching

End of Greedy Algorithms Guide.

Use the table of contents to jump to any topic. Each topic is self-contained for focused revision.